

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana
w specjalności Informatyka w Biznesie

System śledzenia i reidentyfikacji obiektów w nagraniach z wielu kamer monitoringu
z wykorzystaniem mapy terenu

Natalia Turska

numer albumu 340803

promotor
dr inż. Witold Czajewski

Warszawa 2026

System śledzenia i reidentyfikacji obiektów w nagraniach z wielu kamer monitoringu z wykorzystaniem mapy terenu

Streszczenie

Streszczenie pracy powinno w zwięzły sposób opisywać to czego dotyczy praca, co jest jej celem, jakie przyjęto założenia, co w ramach pracy zrobiono, co przebadano, jakie rozwiązania zaproponowano i jakie wyniki osiągnięto. Jeśli coś sprawiło jakiś problem, można to także ująć w streszczeniu i skomentować czy udało się ów problem rozwiązać i jak, czy też nie (nic w tym złego, jeśli czegoś się nie udało do końca zrobić).

Streszczenie nie powinno być przeładowane, powinno mieć około 200 słów i zawierać najważniejsze informacje na temat pracy i najważniejsze osiągnięcia, nie należy więc podawać szczegółowych wyników czy też opisywać struktury pracy - na to miejsce znajduje się w samej pracy.

Streszczenie zwykle pisze się na samym końcu pracy - z oczywistych względów.

W niniejszej pracy magisterskiej zaprezentowane zostały próby rozwiązania problemu klasyfikacji poziomu zabrudzenia chodników z wykorzystaniem głębokich sieci neuronowych. Wykorzystując dostępny zbiór zdjęć sklasyfikowanych do sześciu klas poziomu zanieczyszczeń, wygenerowano zrównoważone zbiory: sześcioklasowy oraz trzyklasowy. Następnie wybrano trzy sieci różnego rodzaju, aby sprawdzić ich efektywność w rozróżnianiu poszczególnych, często bardzo zbliżonych do siebie zdjęć. Sieć TResNet została wybrana spośród wiodących sieci w rankingach klasyfikacyjnych. Sieć ShuffleNet V2 została wybrana spośród dostępnych implementacji sieci w ramach popularnej biblioteki OpenMMLab. Sieć PMG została wybrana spośród sieci przygotowanych do rozwiązywania specyficznych problemów klasyfikacyjnych dla obrazów o niewielkich różnicach. Wymienione sieci wytrenowano na wygenerowanych zbiorach i osiągnięto dokładność rzędu 80% i 90%, odpowiednio dla problemu sześcioklasowego i trzyklasowego. Najlepszą siecią, w każdym przypadku, okazała się sieć PMG, a wyniki pozostałych sieci były zbliżone.

Słowa kluczowe: istotne, słowa, kluczowe

Object Tracking and Re-Identification System for Multi-Camera Surveillance Footage Using a Map of the Monitored Area

Abstract

This engineering thesis presents an attempt to solve the problem of sidewalk dirt level classification using deep neural networks. Using an available set of images classified into six classes of dirt levels, balanced sets of six classes and three classes were generated. Three networks of different types were then selected to test their effectiveness in discriminating between individual, often very similar, images. TResNet was selected among the leading networks in the classification rankings. The ShuffleNet V2 network was selected from available network implementations within the popular OpenMMLab library. The PMG network was selected from networks prepared to solve specific classification problems for images with small differences. The aforementioned networks were trained on the generated sets and an accuracy of 80% and 90% was achieved for the six-class and three-class problem, respectively. The PMG network proved to be the best network, in each case, and the results of the other networks were similar.

Keywords: keywords, that, are, indicative

Spis treści

1	Wstęp	1
2	Stan wiedzy	3
2.1	Analiza obrazu i podstawowe definicje	3
2.1.1	Komputerowa reprezentacja obrazu	3
2.1.2	Model kamery i rzutowanie na plan terenu	4
2.1.3	Detekcja ruchu i odejmowanie tła	5
2.1.4	KNN (<i>ang. K-nearest Neighbors</i>)	8
2.1.5	Ocena efektywności algorytmów	10
2.1.6	Analiza porównawcza i ewaluacja efektywności	11
2.2	Detekcja obiektów oparta na głębokim uczeniu (Deep Learning)	12
2.2.1	Splotowe sieci neuronowe (CNN) i detektory dwuetapowe	13
2.2.2	Detektory jednoetapowe	13
2.2.3	Transformery w detekcji obiektów	14
2.2.4	Dane treningowe a problem luki domenowej	15
2.2.5	Detekcja w trudnych warunkach	16
2.3	Śledzenie wielu obiektów w pojedynczej kamerze	17
2.3.1	Paradygmat śledzenia ścieżek (<i>ang. tracking-by-detection</i>)	17
2.3.2	Metody predykcji stanu i algorytmy skojarzenia	18
2.3.3	Ewolucja kluczowych algorytmów śledzenia	18
2.3.4	Ocena efektywności algorytmów MOT	19
2.3.5	Standardowe benchmarki (MOTChallenge)	20
2.4	Systemy wielokamerowe - MTMC	20
2.4.1	Wprowadzenie do problematyki wielokamerowego śledzenia obiektów	20
2.4.2	Standardowa architektura potoku przetwarzania danych MTMC	21
2.4.3	Problem rzutowania geometrycznego	22
2.4.4	Matematyczne sformułowanie algorytmu POM	23
2.5	Reidentyfikacja obiektów - Re-ID	24
2.5.1	Definicja i sformułowanie problemu reidentyfikacji	24
2.5.2	Klasyczne podejścia oparte na cechach projektowanych ręcznie	25
2.5.3	Uczenie reprezentacji metrycznych i sieci syjamskie	26
2.5.4	Architektury splotowe (CNN) dedykowane do zadań Re-ID	27

2.5.5	Architektury oparte na Transformerach (SOTA)	27
2.5.6	Problem rozbieżności dziedzin i adaptacja bez nadzoru (UDA)	28
2.5.7	Specyfika reidentyfikacji pojazdów w odniesieniu do osób	29
2.5.8	Prywatność i alternatywne metody identyfikacji	29
2.6	Rzutowanie na plan terenu (Bird's-Eye View - BEV) i geometria sceny	30
2.6.1	Geometryczne rzutowanie na plan terenu i Inverse Perspective Mapping	30
2.6.2	Siatka BEV i probabilistyczne mapy zajętości	31
2.6.3	Widok z lotu ptaka w systemach autonomicznej jazdy	31
2.6.4	Metody BEV oparte na głębokim uczeniu	32
2.6.5	Zastosowanie widoku z góry w systemach wielokamerowych MTMC	32
3	Przegląd istniejących rozwiązań	35
3.1	Specyfika sprzętu oraz środowiska programistycznego	35
3.1.1	Architektura sprzętowa: Apple Silicon M5 a akceleracja obliczeń tensorowych	35
3.1.2	Środowisko programistyczne: Język Python oraz Miniforge	36
3.2	Śledzenie wewnątrz pojedynczej kamery i detekcja obiektów	37
3.2.1	Ekosystem YOLO oraz modele RT-DETR	37
3.2.2	Ograniczenia nowszych architektur YOLO w środowisku Apple MPS	38
3.2.3	Asocjacja danych i metody filtracji trajektorii: BoT-SORT i ByteTrack	39
3.3	Reidentyfikacja obiektów (Re-ID)	39
3.3.1	Vision Transformers i TransReID	39
3.3.2	Wybór metod adaptacji domenowej i optymalizacji przestrzeni cech	40
3.3.3	Technologie odrzucone w zadaniu reidentyfikacji	41
3.4	Geometria sceny, rzutowanie BEV i modelowanie tła poprzez bibliotekę OpenCV	42
3.4.1	OpenCV z optymalizacjami dla architektury ARM	42
4	Implementacja systemu śledzenia i reidentyfikacji	45
4.1	Architektura potoku przetwarzania danych	45
4.1.1	Środowisko uruchomieniowe i optymalizacja dla układu Apple M5	45
4.1.2	Wielowątkowość i asynchroniczna wymiana komunikatów	46
4.2	Moduł detekcji i śledzenia w pojedynczej kamerze (SCT)	46
4.2.1	Inicjalizacja i wnioskowanie modelu YOLO	47
4.2.2	Śledzenie lokalne z wykorzystaniem algorytmu BoT-SORT	47
4.3	Ekstrakcja cech wizualnych i reidentyfikacja (Re-ID)	47
4.3.1	Implementacja architektury Vision Transformer (ViT)	47
4.3.2	Generowanie i normalizacja wektorów cech (Embeddings)	48
4.4	Globalna asocjacja danych (Multi-Camera Association)	49
4.4.1	Konstrukcja macierzy kosztów i łączenie błędów	49
4.4.2	Dopasowanie z użyciem algorytmu węgierskiego	49
4.4.3	Zarządzanie cyklem życia globalnych identyfikatorów	49
5	Integracja widoku wielokamerowego i rzutowanie na mapę	51

5.1	Wyznaczanie i aplikacja macierzy homografii	51
5.1.1	Model geometryczny i odwrotne mapowanie perspektywiczne (IPM)	51
5.2	Ograniczenia rzutowania i identyfikacja problemu okluzji	52
5.2.1	Zjawisko błędu paralaksy w gęstym ruchu	52
5.3	Modelowanie tła i ekstrakcja fizycznego ruchu	52
5.3.1	Inicjalizacja algorytmu mieszanin gaussowskich (MOG2)	53
5.3.2	Przetwarzanie morfologiczne masek binarnego ruchu	53
5.4	Implementacja fuzji probabilistycznej mapy zajętości (POM)	53
5.4.1	Generowanie rzutów syntetycznych (Obraz A)	54
5.4.2	Mechanizm weryfikacji karania za niezgodność (Operacja XOR)	54
5.5	Fuzja w module asocjacji i globalna mapa BEV	54
5.5.1	Kalkulacja kosztu przestrzennego	55
5.5.2	Renderowanie ostatecznej, zunifikowanej przestrzeni MTMC	55
6	Badanie i analiza efektywności systemu	57
7	Podsumowanie i wnioski	59
	Bibliografia	61
	Wykaz skrótów i symboli	69
	Spis rysunków	71
	Spis tabel	73

Rozdział 1

Wstęp

Rozdział 2

Stan wiedzy

Najwcześniejsze systemy telewizji przemysłowej (*ang. Closed Circuit Television – CCTV*) powstały w pierwszej połowie XX wieku i umożliwiały jedynie bieżącą obserwację zdarzeń, ze względu na brak technologii pozwalających na rejestrowanie i przechowywanie sygnału wideo. Rozwój nośników szpulowych umożliwił rejestrowanie materiału z monitoringu. Systemy te wymagały były jednak uciążliwe w obsłudze - wymagały ręcznej wymiany taśm magnetycznych oraz ręcznego nawijania taśmy na szpulę odbiorczą. Wraz z postępem technologicznym metody zapisu ulegały systematycznej poprawie, co przyczyniło się do popularyzacji systemów bezpieczeństwa. Już w latach 90. XX opracowano technologię multipleksowania cyfrowego, umożliwiającą jednoczesne nagrywanie z kilku kamer, a także nagrywanie w trybie poklatkowym i zapis wyłącznie przy wykryciu ruchu [33].

Istotny przełom w rozwoju systemów wideonadzoru przyniosła wizja komputerowa, rozwijana od lat 70. XX wieku, której nadrzędnym celem stało się pełne zautomatyzowane zrozumienie analizowanej sceny. Prawdziwą rewolucję w tej dziedzinie umożliwiło jednak dopiero znaczne zwiększenie mocy obliczeniowej, rozwój algorytmów uczenia maszynowego oraz powstanie wielkich, oznaczonych zbiorów danych (m.in. ImageNet, Microsoft COCO czy LVIS). Bazy te dostarczyły niezbędnych informacji do trenowania zaawansowanych modeli oraz pozwoliły na obiektywne mierzenie postępów w dziedzinie rozpoznawania i segmentacji obrazu [61].

2.1 Analiza obrazu i podstawowe definicje

2.1.1 Komputerowa reprezentacja obrazu

Komputer, analizując obraz, sprowadza go do postaci macierzy pikseli, którym przypisane są wartości liczbowe zależne od koloru. Kształt tej macierzy zależy od przestrzeni barw, w jakiej reprezentowany jest dany obraz. W przypadku skali szarości obraz jest przedstawiany w postaci macierzy dwuwymiarowej, której elementy przyjmują wartości od 0 do 255, gdzie 0 oznacza czerń, a 255 biel [23].

Obrazy przedstawione w pełnym spektrum kolorów zapisywane są najczęściej w przestrzeni RGB (*ang. Red, Green, Blue*) lub HSV (*ang. Hue, Saturation, Value*). W takim przypadku obraz opisywany

jest za pomocą trzech macierzy, a każdemu pikselowi przypisywane są trzy wartości z zakresu od 0 do 255. W przestrzeni RGB, która jest wykorzystywana częściej, wartości te odpowiadają odpowiednio natężeniu koloru czerwonego, zielonego i niebieskiego [23]. Takie przedstawienie pozwala na odwzorowanie pełnego spektrum barw w formie dogodnej do przetwarzania komputerowego.

Liczba pikseli tworzących obraz jest równoznaczna z jego rozdzielczością[23], a więc bezpośrednio wpływa na poziom szczegółowości widocznych obiektów i możliwości ich poprawnej detekcji. Nagrania wideo składają się z pojedynczych obrazów nazywanych w tym kontekście klatką lub ramką (*ang. frame*). Ciąg kolejnych klatek wyświetlanych w krótkich odstępach czasu tworzy wrażenie ruchu i stanowi strumień wideo. Liczba klatek na sekundę, wpływa na płynność nagrania oraz możliwość dokładnej analizy ruchu obiektów w czasie.

2.1.2 Model kamery i rzutowanie na plan terenu

Dla poprawnej analizy nagrań wideo należy wziąć pod uwagę sposób ich powstawania w kamerze. Aby możliwe było poprawne wyznaczanie położenia obiektów w scenie, stosuje się proces kalibracji kamery, który pozwala określić zależność między współrzędnymi punktów w przestrzeni rzeczywistej a ich położeniem w obrazie. Podstawą takiego opisu jest model otworkowy kamery (*ang. pinhole camera model*), w którym obraz powstaje jako rzut perspektywiczny punktów trójwymiarowej sceny na dwuwymiarową płaszczyznę obrazu[24].

Pełny model kamery można zapisać w postaci iloczynu macierzy wewnętrznej oraz macierzy zewnętrznej. Macierz wewnętrzna opisuje właściwości samej kamery i sposób odwzorowania punktów w układzie kamery na współrzędne obrazu. Zawiera ona między innymi ogniskowe w osiach poziomej i pionowej, położenie punktu głównego oraz ewentualny parametr skośności osi. Parametry te określają więc geometryczne cechy obrazowania i wpływają na skalę oraz położenie obrazu w układzie pikseli[61].

Grupa parametrów zewnętrznych, opisuje położenie i orientację kamery względem układu współrzędnych sceny. Obejmują one macierz obrotu oraz wektor translacji. Parametry te opisują transformację punktów ze współrzędnych świata do układu współrzędnych kamery. Dzięki temu możliwe jest określenie, z którego miejsca kamera obserwuje scenę oraz w jakim kierunku jest skierowana[61].

Złożenie macierzy wewnętrznej i zewnętrznej daje pełną macierz projekcji kamery $P = K[R \mid t]$, która opisuje kompletną transformację punktu z trójwymiarowej przestrzeni sceny do dwuwymiarowego układu pikselowego obrazu [24].

W praktyce rzeczywisty obraz może odbiegać od idealnego modelu z powodu dystorsji soczewki, która powoduje zniekształcenia, są one szczególnie widoczne na obrzeżach obrazu i wynikają z właściwości układu optycznego. Z tego względu w procesie kalibracji często wyznacza się również współczynniki pozwalające na korektę tych zniekształceń przed dalszą analizą obrazu[61].

Do analizy wielu obrazów tej samej powierzchni lub terenu służy analiza homografii. W dziedzinie widzenia komputerowego dowolne dwa obrazy tej samej płaskiej powierzchni w przestrzeni są

powiązane przekształceniem rzutowym, zwanym właśnie homografią. Opisuje się ją za pomocą macierzy H o wymiarach 3×3 , która odwzorowuje punkty z jednej płaszczyzny na drugą z uwzględnieniem zniekształceń perspektywy. Macierz ta działa na punkty zapisane we współrzędnych jednorodnych i przyjmuje postać:

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

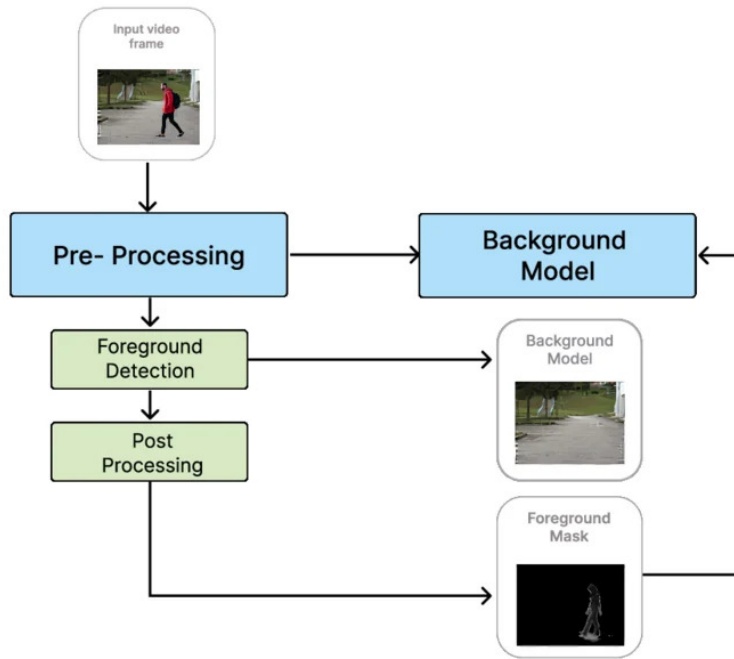
gdzie (x, y) oznacza współrzędne punktu w pierwszym układzie, (x', y') współrzędne punktu po przekształceniu, natomiast λ jest współczynnikiem skali wynikającym z użycia współrzędnych jednorodnych. Taki zapis pozwala ująć w jednej macierzy przesunięcie, obrót, zmianę skali oraz zniekształcenie perspektywiczne [74, 61].

W kontekście systemów wielokamerowych rzutowanie na plan terenu umożliwia przedstawienie trajektorii obiektów w jednej, wspólnej przestrzeni geometrycznej, niezależnie od położenia i orientacji poszczególnych kamer.

Należy jednak zaznaczyć, że poprawność takiego rzutowania zależy od jakości kalibracji kamery oraz od tego, w jakim stopniu obserwowana scena spełnia założenie o płaskości terenu. W przypadku nierównego podłoża lub obiektów o znacznej wysokości konieczne jest zastosowanie bardziej rozbudowanych modeli geometrycznych.

2.1.3 Detekcja ruchu i odejmowanie tła

Technika odejmowania tła (*ang. background subtraction*) umożliwia rozpoznanie ruchomych obiektów na nagraniach wideo dla statycznych scen poprzez oddzielenie pierwszego planu od tła. Przyjmując założenie, że tło pozostaje względnie niezmiennie (mogą pojawiać się zmiany oświetlenia, zabrudzenia itp.), możliwe jest opisanie tła za pomocą modelu statystycznego. Stworzony model tła odejmowany jest od obrazu, pozostawiając jedynie anomalie, czyli obiekty znajdujące się w ruchu, pierwszy plan zwracany jest zazwyczaj w postaci maski.[80]



Rysunek 1. Wizualizacja procesu odejmowania tła. Źródło: [60]

W kontekście projektowania zaawansowanych systemów śledzenia wielu obiektów i reidentyfikacji, algorytmy odejmowania tła pozwalają na ograniczenie obszaru poszukiwań, system może przetwarzać wyłącznie wykadrowane obszary ruchu (ang. Regions of Interest – ROI).

MOG2 (Mixture of Gaussians 2)

Algorytm MOG2 stanowi bezpośrednie rozwinięcie metody mieszanin rozkładów Gaussa zaproponowanej przez Stauffera i Grimsona. Model GMM (ang. *Gaussian Mixture Model*) zakłada stałą liczbę komponentów gaussowskich dla każdego piksela w scenie, co może prowadzić do nadmiernej złożoności obliczeniowej dla jednorodnych i statycznych obszarów obrazu. MOG2, opracowany przez Zivkovic, wprowadza adaptacyjny dobór liczby rozkładów dla każdego piksela z osobna.[80]

W algorytmie MOG2 prawdopodobieństwo zaobserwowania określonej wartości piksela $\vec{x}$ w chwili t w wielowymiarowej przestrzeni barw (np. RGB) jest opisywane przez ważoną sumę M rozkładów normalnych [58, 80]:

$$\hat{p}(\vec{x} | X_T, BG + FG) = \sum_{m=1}^M \hat{\pi}_m \mathcal{N}(\vec{x}; \hat{\vec{\mu}}_m, \hat{\sigma}_m^2 \mathbf{I}) \quad (1)$$

gdzie $\hat{\pi}_m$ oznacza oszacowaną wagę m -tego komponentu, $\hat{\vec{\mu}}_m$ jest jego wektorem średnich, natomiast $\hat{\sigma}_m^2 \mathbf{I}$ reprezentuje macierz kowariancji [58, 80]. Dla uproszczenia obliczeń zakłada się, że poszczególne kanały kolorów są od siebie niezależne i posiadają tę samą wariancję, co sprowadza

macierz kowariancji do iloczynu wariancji $\hat{\sigma}_m^2$ oraz macierzy jednostkowej I [58, 80]. Wagi mieszaniny są nieujemne i podlegają normalizacji, spełniając warunek $\sum_{m=1}^M \hat{\pi}_m = 1$ [58, 80].

Dla każdej nowej klatki obrazu i każdej nowo zaobserwowanej wartości piksela $\vec{x}(t)$ realizowana jest rekurencyjna aktualizacja parametrów modelu [58, 80]. Najpierw następuje porządkowanie komponentów według stosunku wagi do odchylenia standardowego ($\hat{\pi}_m/\hat{\sigma}_m$), co faworyzuje rozkłady o wysokiej wadze i niskiej wariancji jako najbardziej prawdopodobne definicje tła [58]. Dopasowanie próbki do istniejących komponentów określa się na podstawie odległości Mahalanobisa [58, 80]. Jeśli kwadrat tej odległości jest mniejszy od przyjętego progu (odpowiadającego zazwyczaj trzem odchyleniom standardowym), komponent uznaje się za dopasowany [58]. Zmienna przynależności $o_m^{(t)}$ przyjmuje wówczas wartość 1 dla najlepiej dopasowanego rozkładu i 0 dla pozostałych [58]. Aktualizacja parametrów zachodzi według następujących równań rekurencyjnych [58, 80]:

$$\hat{\pi}_m \leftarrow \hat{\pi}_m + \alpha(o_m^{(t)} - \hat{\pi}_m) \quad (2)$$

$$\hat{\mu}_m \leftarrow \hat{\mu}_m + o_m^{(t)} \left(\frac{\alpha}{\hat{\pi}_m} \right) \vec{\delta}_m \quad (3)$$

$$\hat{\sigma}_m^2 \leftarrow \hat{\sigma}_m^2 + o_m^{(t)} \left(\frac{\alpha}{\hat{\pi}_m} \right) \left(\vec{\delta}_m^T \vec{\delta}_m - \hat{\sigma}_m^2 \right) \quad (4)$$

gdzie $\vec{\delta}_m = \vec{x}(t) - \hat{\mu}_m$, natomiast α określa współczynnik decydujący o tempie zmian parametrów rozkładu, $\alpha = 1/T$, gdzie T jest rozważanym interwałem czasowym (liczbą klatek) [58, 81]. Jeśli dla danego piksela nie zostanie znaleziony żaden dopasowany komponent, tworzony jest nowy rozkład o parametrach: $\hat{\pi}_{M+1} = \alpha$, $\hat{\mu}_{M+1} = \vec{x}(t)$ oraz wariancji początkowej $\hat{\sigma}_{M+1}^2 = \sigma_0^2$ [58]. Jeśli liczba komponentów przekroczy dopuszczalne maksimum, rozkład o najmniejszej wadze jest usuwany [58].

Ujemny rozkład a priori Dirichleta i eliminacja komponentów

W celu automatycznego określenia optymalnej liczby komponentów M dla każdego piksela, Zivkovic wprowadził do estymacji parametrów metodę maksymalizacji prawdopodobieństwa a posteriori (MAP) z użyciem sprzężonego rozkładu a priori Dirichleta [80, 81]. Zdefiniowano rozkład a priori w postaci [80]:

$$P = \prod_{m=1}^M \pi_m^{c_m} \quad (5)$$

Kluczowym elementem tej metody jest nadanie współczynnikom rozkładu a priori wartości ujemnych ($c_m = -c$) [80]. Z matematycznego punktu widzenia ujemne współczynniki wprowadzają karę za nadmierną złożoność modelu, co jest ściśle powiązane z kryterium minimalnej długości opisu (ang. *Minimum Message Length* – MML) [80]. Rozwiązanie równania MAP przy uwzględnieniu ujemnego rozkładu a priori prowadzi do wyznaczenia wag skorygowanych [80]:

$$\hat{\pi}_m^{(t)} = \frac{1}{K} \left(\sum_{i=1}^t o_m^{(i)} - c \right) \quad (6)$$

gdzie parametr normalizacyjny wynosi $K = t - Mc$ [80]. Aby zapobiec zanikaniu wpływu kary wraz z upływem czasu (gdy $t \rightarrow \infty$), ułamek c/t zastępuje się stałą wartością $c_T = c/T$ [80]. Przy założeniu, że rzeczywista liczba komponentów opisujących piksel jest niewielka, ostateczne równanie aktualizacji wag przyjmuje postać [80]:

$$\hat{\pi}_m \leftarrow \hat{\pi}_m + \alpha(o_m^{(t)} - \hat{\pi}_m) - \alpha c_T \quad (7)$$

Zastosowanie stałej poprawki ujemnej $-\alpha c_T$ sprawia, że wagi komponentów, które nie są regularnie wspierane przez nowe obserwacje, sukcesywnie maleją [80]. Gdy waga danego komponentu osiągnie wartość ujemną, zostaje on trwale usunięty z mieszaniny [80]. W ten sposób piksele reprezentujące stabilne elementy tła są opisywane zaledwie jednym rozkładem Gaussa, natomiast piksele w obszarach o złożonej dynamice (np. migoczące światła, falująca roślinność) utrzymują większą liczbę komponentów [80, 81].

2.1.4 KNN (*ang. K-nearest Neighbors*)

Algorytm KNN w kontekście odejmowania tła stanowi alternatywne, nieparametryczne podejście do modelowania rozkładu prawdopodobieństwa pikseli. W podejściu nieparametrycznym funkcja gęstości jest szacowana bezpośrednio na podstawie danych historycznych, bez zakładania konkretnego kształtu rozkładu [16]. Podstawową techniką wykorzystywaną w tym podejściu jest estymacja gęstości jądrowej (KDE, *ang. Kernel Density Estimation*), która wymaga przechowywania zbioru N próbek historycznych $\vec{x}_n$ reprezentujących wcześniej obserwowane wartości pikseli [16].

Estymacja gęstości jądrowej z użyciem estymatora balonowego

Podstawowy estymator Parzena–Rosenblatta szacuje gęstość prawdopodobieństwa punktu $\vec{x}$ w d -wymiarowej przestrzeni cech na podstawie N próbek historycznych $\vec{x}_n$ jako:

$$\hat{p}(\vec{x}) = \frac{1}{Nh^d} \sum_{n=1}^N K\left(\frac{\vec{x} - \vec{x}_n}{h}\right) \quad (8)$$

gdzie $K(\cdot)$ jest funkcją jądrową, a h stałą szerokością pasma. W kontekście odejmowania tła szerokość pasma decyduje o wrażliwości klasyfikacji: jeżeli piksel znajduje się poza promieniem wyznaczonym przez jądro, zostaje zaklasyfikowany jako pierwszy plan [81]. Użycie stałej szerokości pasma może generować błędy w obszarach o zróżnicowanej gęstości danych — nadmierne wygładzenie w regionach gęstych i niedoszacowanie w rzadkich [63].

Zastosowanie estymatora balonowego (*ang. balloon estimator*) [63] rozwiązuje ten problem poprzez uzależnienie szerokości jądra h od lokalnej gęstości w punkcie estymacji $\vec{x}$. Szerokość okna dobiera się odwrotnie proporcjonalnie do lokalnej gęstości:

$$h(\vec{x}) = \frac{k}{[N \hat{p}(\vec{x})]^{1/d}} \quad (9)$$

Wówczas estymowana gęstość prawdopodobieństwa wyraża się wzorem:

$$\hat{p}(\vec{x}) \approx \frac{1}{N \cdot h(\vec{x})^d} \sum_{n=1}^N K\left(\frac{\vec{x} - \vec{x}_n}{h(\vec{x})}\right) \quad (10)$$

Dla optymalizacji wydajności obliczeniowej w systemach czasu rzeczywistego jako funkcję jądrową $K(u)$ stosuje się jądro jednostajne (prostokątne) [63]. Wówczas estymacja gęstości sprowadza się do zliczenia próbek $\vec{x}_n$ leżących wewnątrz sfery o promieniu $h(\vec{x})$ wyznaczonym przez odległość do k -tego sąsiada [81]:

$$\hat{p}(\vec{x}) \approx \frac{k}{N \cdot V_d \cdot h(\vec{x})^d} \quad (11)$$

gdzie V_d oznacza objętość jednostkowej kuli w przestrzeni d -wymiarowej. Takie podejście drastycznie skraca czas obliczeń w porównaniu z jądrem gaussowskim, przy zachowaniu adaptacyjności szerokości pasma [81, 44].

Metoda k-najbliższych sąsiadów w przestrzeni barw RGB

W praktycznej implementacji algorytmu KNN klasyfikacja nowego piksela $\vec{x}(t)$ opiera się na następującym schemacie decyzyjnym [81]:

1. Zakłada się, że piksel jest reprezentowany jako wektor w przestrzeni barw RGB: $\vec{x}(t) = [R(t), G(t), B(t)]^T$.
2. Oblicza się odległości euklidesowe d_n między $\vec{x}(t)$ a każdą z N próbek zgromadzonych w buforze historycznym tego piksela $\mathcal{X} = \{\vec{x}_1, \dots, \vec{x}_N\}$ [81, 13].
3. Zlicza się liczbę próbek k^* , dla których d_n jest mniejsza od progu D_{thr} (w OpenCV parametr `dist2Threshold` przechowuje kwadrat tej odległości):

$$k^* = \sum_{n=1}^N 1[d(\vec{x}(t), \vec{x}_n) < D_{thr}] \quad (12)$$

gdzie $1[\cdot]$ oznacza funkcję wskaźnikową. Próbkę spełniającą ten warunek mieszczą się wewnątrz sfery o promieniu D_{thr} wokół $\vec{x}(t)$ [50, 81, 47].

4. Jeśli $k^* \geq k$, piksel zostaje zaklasyfikowany jako tło [81, 47]. W przeciwnym razie jest oznaczany jako pierwszy plan [81, 47].

Aktualizacja modelu w algorytmie KNN zachodzi w sposób ciągły — nowa próbka piksela zastępuje najstarszą próbkę w buforze (schemat FIFO). Dzięki temu model stopniowo dostosowuje się do wolnych zmian warunków oświetleniowych, takich jak ruch chmur czy dynamika cieni [81].

2.1.5 Ocena efektywności algorytmów

Wyniki segmentacji pierwszego planu weryfikuje się przez porównanie wygenerowanej maski binarnej z maską referencyjną (*ang. ground truth*) na poziomie pikseli [68]. Każdy piksel klasyfikowany jest jako jeden z czterech przypadków: prawdziwy pozytywny (TP – piksel pierwszego planu wykryty poprawnie), fałszywy pozytywny (FP – piksel tła błędnie oznaczony jako pierwszy plan), prawdziwy negatywny (TN – piksel tła wykryty poprawnie) oraz fałszywy negatywny (FN – piksel pierwszego planu pominięty przez algorytm). Na tej podstawie wyznacza się następujące metryki [68]:

- **Czułość** (*ang. Recall / True Positive Rate*) — stosunek poprawnie wykrytych pikseli pierwszego planu do wszystkich pikseli pierwszego planu w masce referencyjnej:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (13)$$

- **Swoistość** (*ang. Specificity / True Negative Rate*) — stosunek poprawnie sklasyfikowanych pikseli tła do wszystkich pikseli tła w masce referencyjnej:

$$\text{Specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} \quad (14)$$

- **Precyzja** (*ang. Precision*) — stosunek poprawnie wykrytych pikseli pierwszego planu do wszystkich pikseli oznaczonych przez algorytm jako pierwszy plan:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (15)$$

- **Miara F1** (*ang. F1-score*) — średnia harmoniczna precyzji i czułości, zapewniająca zrównoważoną ocenę przy nierównomiernym rozkładzie klas:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (16)$$

- **IoU** (*ang. Intersection over Union*) — stosunek pola przecięcia maski wykrytej i referencyjnej do pola ich sumy. W kontekście ewaluacji BGS operuje się na maskach binarnych, nie na ramkach otaczających [68]:

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \quad (17)$$

- **PWC** (*ang. Percentage of Wrong Classifications*) — odsetek błędnie sklasyfikowanych pikseli względem całkowitej liczby pikseli w sekwencji; stosowany jako metryka syntetyczna

w benchmarku CDnet [68]:

$$PWC = 100 \cdot \frac{FP + FN}{TP + FP + TN + FN} \quad (18)$$

Wymienione metryki są standardowo stosowane w benchmarku CDnet 2014 [68], który stanowi punkt odniesienia dla porównania algorytmów BGS w niniejszej pracy. Metryki oceny śledzenia wielu obiektów (MOTA, HOTA, IDF1) oraz reidentyfikacji (Rank-1/CMC) zostają omówione w kolejnych podrozdziałach poświęconych odpowiednio modułom detekcji i śledzenia.

2.1.6 Analiza porównawcza i ewaluacja efektywności

Porównanie wydajności algorytmów BGS w trudnych warunkach atmosferycznych

Efektywność algorytmów odejmowania tła zależy od charakterystyki badanej sceny oraz rodzaju zakłóceń środowiskowych [7]. Tabela 1 przedstawia średnie wartości współczynnika IoU uzyskane przez wybrane algorytmy BGS w badaniach porównawczych dla różnego rodzaju zakłóceń atmosferycznych wideo [31].

Tabela 1. Średnie wartości współczynnika IoU wybranych algorytmów BGS. Źródło: opracowanie własne na podstawie danych z [31].

Środowisko / Warunki	CNT	KNN	MOG	MOG2	GMG	SOM
Śnieg	0,41	0,38	0,44	0,25	0,42	0,52
Wiatr	0,13	0,13	0,33	0,11	0,12	0,29
Deszcz	0,17	0,14	0,37	0,10	0,31	0,25
Średnia	0,28	0,32	0,39	0,25	0,42	0,47

Analiza wyników przedstawionych w Tabeli 1 pozwala na sformułowanie kilku obserwacji dotyczących zachowania algorytmów BGS w trudnych warunkach atmosferycznych.

Najślabszą i najbardziej równomierną odporność na zakłócenia środowiskowe wykazują algorytmy KNN oraz MOG2 – oba uzyskują najniższe wartości IoU zarówno przy wietrze (0,13 i 0,11), jak i przy deszczu (0,14 i 0,10). Algorytm MOG z kolei wykazuje zaskakująco wysoką odporność w warunkach wietrznych (IoU = 0,33) oraz deszczowych (IoU = 0,37), co wynika z jego większej bezwładności modelu – stała liczba komponentów gaussowskich wolniej absorbuje szybkie zakłócenia [58].

Spośród wszystkich badanych metod najwyższą średnią skuteczność osiąga SOM (IoU = 0,47), a najniższą – MOG2 (IoU = 0,25). Algorytm SOM (*ang. Self-Organizing Map*) modeluje tło za pomocą samoorganizacji poprzez szczutyczną sieć neuronową [31]. Dzięki temu skuteczniej radzi sobie ze złożonymi problemami, takimi jak ruszające się tło czy zmiany w oświetleniu [31].

Algorytm GMG osiąga wysoką średnią (IoU = 0,42) dzięki bayesowskiemu modelowi tła budowanemu na podstawie pierwszych klatek sekwencji (domyślnie 120 klatek w OpenCV). Wysoka skuteczność przy śniegu (IoU = 0,42) jest efektem dobrego odwzorowania statystycznej zmienności tła w fazie inicjalizacji. Metoda ta ma jednak istotne ograniczenie – wymaga stabilnej fazy uczenia i słabo radzi sobie z dynamicznymi scenami od pierwszej klatki [31].

Algorytm CNT (*ang. Counter, ang. Count of Non-stationary Targets*) opiera się na zliczaniu, ile razy wartość piksela zmieniła się w danym czasie. Jest to metoda prosta obliczeniowo, osiągająca zadowalające wyniki dla śniegu ($IoU = 0,41$), jest zawodna przy wietrze i deszczu ($IoU = 0,13$ i $0,17$).

Pomimo że algorytmy SOM, GMG i CNT uzyskują wyższe wartości IoU od KNN i MOG2, w niniejszej pracy zdecydowano się na implementację tych ostatnich. Wynika to z ich natywnej dostępności i pełnego wsparcia w bibliotece OpenCV, niskiego zapotrzebowania na zasoby obliczeniowe niezbędnego w przetwarzaniu strumienia wideo w czasie rzeczywistym, a także szczegółowej dokumentacji i ugruntowanej pozycji w literaturze [81]. Algorytmy SOM i GMG wymagają odpowiednio dłuższej fazy inicjalizacji lub wyższych zasobów, co utrudnia ich zastosowanie w scenariuszach z dynamicznie zmieniającym się tłem bez etapu wstępnego uczenia.

Wydajność obliczeniowa, ograniczenia sprzętowe i zjawisko dryfu tła

W systemach czasu rzeczywistego wydajność obliczeniowa jest istotnym czynnikiem przy wdrażaniu oprogramowania[7]. MOG2, dzięki dynamicznemu doborowi liczby komponentów gaussowskich opartemu na kryterium MML (*ang. Minimum Message Length*) [80], charakteryzuje się niskim zużyciem pamięci RAM. Pozwala to na jego stosowanie w architekturach wielokamerowych na platformach wbudowanych [65].

Najpoważniejszą wadą klasycznych algorytmów BGS, w tym MOG2 i KNN, jest podatność na zjawisko dryfu modelu tła (*ang. background drift*) [65]. Jeżeli obiekt pierwszego planu pozostanie nieruchomy, w ciągu 100 klatek może zostać zaklasyfikowany jako część tła [65]. Skutkuje to fragmentacją ścieżki śledzenia i utratą ciągłości detekcji. Dlatego na trudnych zbiorach testowych samodzielne algorytmy BGS osiągają niską czułość detekcji [65].

2.2 Detekcja obiektów oparta na głębokim uczeniu (Deep Learning)

Przed upowszechnieniem wielowarstwowych sieci neuronowych, systemy detekcji obiektów opierały się na ręcznie projektowanych cechach wizualnych (*ang. hand-crafted features*), które były następnie klasyfikowane za pomocą tradycyjnych algorytmów uczenia maszynowego. Jednym z najważniejszych rozwiązań w obszarze detekcji obiektów w czasie rzeczywistym stał się algorytm Viola-Jones, pierwotnie opracowany do detekcji twarzy. Metoda ta opiera się na trzech kluczowych koncepcjach: reprezentacji obrazu całkowego (*ang. Integral Image*), wyznaczaniu prostych cech (*ang. Haar-like features*) oraz kaskadowej strukturze klasyfikatorów opartej na algorytmie AdaBoost [64]. Zastosowanie obrazu całkowego umożliwiło obliczanie sum jasności pikseli wewnątrz prostokątnych obszarów w czasie stałym $O(1)$, niezależnie od skali. Pomimo rewolucyjnej szybkości działania, detektor Viola-Jones wykazywał niską odporność na rotacje obiektów w przestrzeni trójwymiarowej, okluzyje oraz gwałtowne zmiany kontrastu i oświetlenia sceny [64]. Kolejny przełomowy algorytm, służący do detekcji sylwetek ludzkich, to metoda HOG + SVM. Algorytm opierał się na histogramach zorientowanych gradientów liczonych w komórkach obrazu i normalizowanych lokalnie w nakładach

jących się blokach, co poprawiało odporność na zmiany kontrastu [14]. Autorzy wykazali, że taka reprezentacja znacznie przewyższa wcześniejsze zestawy cech w zadaniu detekcji ludzi. Ograniczeniem podejścia pozostawała m.in. podatność na deformacje postawy oraz koszt obliczeniowy wynikający ze skanowania obrazu metodą przesuwanego okna [14].

2.2.1 Splotowe sieci neuronowe (CNN) i detektory dwuetapowe

Rok 2012 przyniósł zasadnicze zmiany w wizji komputerowej wraz z sukcesem głębokiej sieci splotowej AlexNet w konkursie ILSVRC [32]. Model ten, zbudowany z pięciu warstw splotowych i trzech warstw w pełni połączonych, pokazał, że sieci neuronowe mogą samodzielnie uczyć się użytecznych cech obrazu lepiej niż metody ręcznie projektowane [32]. Zastosowanie funkcji ReLU oraz technik regularizacji, takich jak dropout, umożliwiło skuteczne trenowanie dużych modeli na procesorach graficznych (GPU).

Zastosowanie sieci splotowych do lokalizacji obiektów doprowadziło do powstania detektorów dwuetapowych. W pierwszym kroku system wskazywał możliwe miejsca występowania obiektu, a w drugim klasyfikował je i doprecyzowywał położenie. Pierwszym istotnym modelem był R-CNN, który łączył tradycyjne generowanie propozycji regionów, na przykład metodą Selective Search, z siecią CNN analizującą każdy region osobno [22]. Było to jednak wolne, ponieważ ten sam fragment obrazu był przetwarzany wiele razy. Fast R-CNN przyspieszył ten proces, obliczając cechy dla całego obrazu tylko raz i wykorzystując warstwę ROI Pooling do pobierania informacji dla proponowanych regionów [21]. Dalszym ulepszeniem był Faster R-CNN, który zastąpił Selective Search siecią RPN, czyli modułem proponującym regiony bezpośrednio na podstawie map cech [53]. Dzięki temu model stał się znacznie szybszy i bardzo dokładny, choć nie zawsze osiągał pełny czas rzeczywisty.

2.2.2 Detektory jednoetapowe

Modele dwuetapowe, mimo wysokiej dokładności, często były zbyt wolne do zastosowań wymagających szybkiej reakcji, takich jak monitoring czasu rzeczywistego czy systemy wbudowane. Odpowiadając na tę potrzebę, powstały detektory jednoetapowe (ang. *one-stage detectors*), które bezpośrednio przekształcają obraz wejściowy w predykcje klas oraz współrzędnych ramek otaczających, bez osobnego etapu generowania propozycji regionów.

Jednym z najważniejszych algorytmów tej grupy stał się YOLO (ang. *You Only Look Once*), w którym potraktowano detekcję obiektów jako pojedynczy problem regresji realizowany przez jedną sieć neuronową w jednym przebiegu obrazu (*single forward pass*) [52]. Dzięki temu możliwe stało się wykrywanie obiektów w czasie zbliżonym do rzeczywistego, przy zachowaniu dobrej skuteczności. Kolejne wersje tej rodziny rozwijały to podejście, poprawiając kompromis między szybkością a dokładnością. YOLOv7 wprowadził architekturę E-ELAN (ang. *Extended Efficient Layer Aggregation Network*), która usprawniła przepływ informacji w sieci, oraz koncepcję *compound scaling*, pozwalającą lepiej dobierać rozmiar modelu do możliwości obliczeniowych. Autorzy zastosowali także tzw. *bag-of-freebies*, czyli zestaw ulepszeń poprawiających jakość uczenia bez zwiększania

kosztu inferencji [66]. YOLOv8 przeszedł na podejście bezkotwicowe (*anchor-free*), rezygnując z wcześniej definiowanych ramek bazowych, oraz zastosował rozdzieloną głowę detekcyjną (*ang. split head*), w której klasyfikacja i regresja są wykonywane osobno [29]. Z kolei YOLOv9 zaproponował rozwiązania mające ograniczać utratę informacji w głębokich sieciach oraz poprawiać przepływ gradientu, co ma szczególne znaczenie w przypadku małych i trudnych do wykrycia obiektów. [67].

Alternatywnym podejściem do detekcji jednoetapowej jest SSD (*ang. Single Shot MultiBox Detector*). Metoda ta wykorzystuje kilka map cech o różnych rozdzielczościach, dzięki czemu lepiej radzi sobie z obiektami o zróżnicowanych rozmiarach [40]. W porównaniu z modelami dwuetapowymi SSD oferował prostszą architekturę i większą szybkość działania, choć początkowo często ustępował im dokładnością.

Głównym problemem detektorów jednoetapowych była jednak silna nierównowaga klasowa: w obrazie dominuje tło, a obiekty zajmują tylko niewielką część danych. Prowadziło to do sytuacji, w której model był zbyt mocno uczony rozpoznawania tła, a zbyt słabo skupiał się na rzeczywistych obiektach [39]. Problem ten skutecznie złagodził RetinaNet, który wprowadził funkcję straty Focal Loss:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

Czynnik modulujący $(1 - p_t)^\gamma$ (gdzie γ to parametr skupienia) sprawia, że w tym ujęciu łatwe przykłady, czyli takie, które model klasyfikuje z dużą pewnością, otrzymują mniejszą wagę w procesie uczenia. Dzięki temu gradient jest bardziej skoncentrowany na trudnych przykładach i na obiektach pierwszoplanowych, co poprawia skuteczność detekcji bez rezygnacji z szybkości charakterystycznej dla podejścia jednoetapowego [39].

2.2.3 Transformery w detekcji obiektów

Wprowadzenie mechanizmów uwagi, znanych wcześniej głównie z przetwarzania języka naturalnego, doprowadziło do powstania architektury DETR (*Detection Transformer*). Model ten traktuje detekcję obiektów jako problem bezpośredniego przewidywania zbioru wyników (*direct set prediction*). W praktyce oznacza to, że zamiast stosować kotwice (*anchors*) i dodatkowy etap usuwania nadmiarowych ramek NMS (*ang. Non-Maximum Suppression*), DETR łączy ekstraktor cech oparty na sieci splotowej z enkoderem i dekoderni Transformer oraz mechanizmem dopasowania dwudzielnego opartym na algorytmie węgierskim [8].

Mimo eleganckiej konstrukcji oryginalny DETR miał istotne ograniczenia. Model uczył się bardzo wolno, wymagał dużych zasobów obliczeniowych i osiągał słabsze wyniki w detekcji małych obiektów. Odpowiedzią na te problemy był Deformable DETR. Zastosowano w nim odkształcalną uwagę (*deformable attention*), która zamiast analizować cały obraz, skupia się na niewielkiej liczbie punktów wybranych wokół punktów referencyjnych. Dzięki temu model lepiej wykorzystuje cechy wieloskalowe, szybciej się uczy i radzi sobie skuteczniej z obiektami o różnych rozmiarach [79].

Nowszym rozwinięciem tej linii jest RT-DETR (*Real-Time Detection Transformer*), zaprojektowany z myślą o zastosowaniach czasu rzeczywistego. Model ten łączy wydajny enkoder hybrydowy

z mechanizmem wyboru najbardziej obiecujących zapytań dekodera, co pozwala ograniczyć koszty obliczeniowe i zachować wysoką szybkość działania. W odróżnieniu od klasycznych detektorów opartych na CNN RT-DETR dąży do osiągnięcia bardzo dobrego kompromisu między dokładnością a czasem inferencji, a przy tym eliminuje potrzebę stosowania NMS jako osobnego etapu przetwarzania [76].

2.2.4 Dane treningowe a problem luki domenowej

Skuteczność głębokich modeli detekcji zależy od wielkości i jakości danych uczących. Modele są zazwyczaj trenowane na dużych, ogólnych zbiorach badawczych, takich jak MS COCO lub ImageNet. Jednak bezpośrednie przeniesienie modelu wytrenowanego na danych syntetycznych do rzeczywistego środowiska przemysłowego lub systemu CCTV zawodzi, ujawniając problem luki domenowej (*ang. domain shift*)[26].

Luka domenowa wynika z różnic w charakterystyce obrazów między zbiorem treningowym a rzeczywistym środowiskiem.

- Zdjęcia w zbiorach akademickich najczęściej wykonywane są z perspektywy na wysokości oczu, kamery monitoringu są zwykle umieszczone znacznie wyżej. Powoduje to zniekształcenia perspektywy, które wpływają na kształt i rozmiar obiektów, a także na ich wzajemne relacje przestrzenne.
- Kamery CCTV charakteryzują się specyficznymi zniekształceniami optycznymi (np. obiektywy szerokokątne typu rybie oko), silną kompresją stratną strumienia wideo oraz wysokim poziomem szumów matrycy w warunkach nocnych.
- Zbiory testowe rzadko reprezentują skrajne zjawiska pogodowe (silny deszcz, śnieżyca, mgła, oślepiające słońce generujące głębokie cienie), które w rzeczywistym monitoringu zewnętrznym występują powszechnie [26].

Najczęściej stosowane techniki niwelowania luki domenowej w detekcji obiektów można podzielić na trzy główne grupy: translację domen, samouczenie z pseudoetykietami oraz dopasowanie cech między domenami. Pierwsza grupa polega na przekształcaniu danych źródłowych tak, aby bardziej przypominały dane docelowe, zwykle przy użyciu modeli generatywnych. Wiąże się to jednak z utratą informacji i wysoką złożonością obliczeniową. [26].

Metody samouczenia (*ang. self-training*), w których model uczony jest na danych źródłowych z etykietami oraz na danych docelowych opisanych pseudoetykietami generowanymi przez model nauczyciela. Jego skuteczność zależy jednak od jakości pseudoetykiet, dlatego wiele nowszych metod wprowadza mechanizmy ich selekcji, ważenia lub filtrowania [26].

Szeroko stosowane metody dopasowania cech (*domain alignment*), dążą do zmniejszenia różnic pomiędzy rozkładami cech w domenie źródłowej i docelowej. W klasycznych rozwiązaniach wykorzystuje się do tego sieci przeciwstawne, w których dyskryminator próbuje odróżnić cechy pochodzące z obu domen, a ekstraktor cech uczy się tworzyć reprezentacje nieodróżnialne z punktu widzenia domeny. W detekcji obiektów takie dopasowanie może zachodzić na poziomie obrazu, regionu lub pojedynczej instancji [10, 26].

Nowsze badania wskazują jednak, że globalne wyrównywanie cech na poziomie całego obrazu szkodzi precyzji lokalizacji detektorów, gdyż prowadzi do transferu informacji z nieistotnych obszarów tła [26]. W związku z tym wprowadzono zaawansowane strategie adaptacji różnicowej (np. UFOA – Uncertainty-based Foreground-Oriented Alignment) [26], które selektywnie nadają wyższy priorytet obszarom reprezentującym obiekty pierwszoplanowe.

2.2.5 Detekcja w trudnych warunkach

Wdrożenie skutecznych algorytmów detekcji obiektów w systemach monitoringu wymaga wysokiej odporności modeli na zakłócenia. Do najczęstszych problemów należą okluzje, gwałtowne zmiany oświetlenia oraz niska rozdzielczość małych lub odległych obiektów [45, 11].

Techniki ograniczające wpływ okluzji obejmują między innymi:

- **Generatywne warstwy kompozycyjne** – w tym podejściu klasyczne głowice klasyfikacyjne zastępowane są strukturą generatywną, która potrafi wnioskować o niewidocznych częściach obiektu na podstawie relacji przestrzennych między jego widocznymi fragmentami. Przykładem takiego rozwiązania jest model CompositionalNet [11].
- **Algorytmy głosowania częściowego (*ang. part-based voting*)** – model uczy się niezależnych reprezentacji poszczególnych części obiektu, a następnie szacuje położenie całego obiektu na podstawie wykrytych elementów [11]. W detekcji pieszych może to oznaczać osobne wykrywanie głowy, ramion i nóg, a następnie wyznaczenie położenia całej sylwetki.
- **Łączenie wielu widoków i wspólne wnioskowanie** – w systemach wielokamerowych obiekt zasłonięty z jednej perspektywy może być dobrze widoczny z innej. Połączenie predykcji z kilku sąsiadujących kamer pozwala trafniej odtworzyć położenie obiektu, nawet jeśli w pojedynczym obrazie jest on silnie zasłonięty [11].

Systemy CCTV działają przez całą dobę, dlatego są narażone na zmienne warunki oświetleniowe utrudniające detekcję. Problem stanowi zarówno intensywne światło słoneczne powodujące głębokie cienie, jak i nagłe prześwielenia, na przykład od reflektorów samochodowych w nocy, a także bardzo słabe oświetlenie po zmroku [45]. Dodatkowo opady atmosferyczne, takie jak deszcz i śnieg, wprowadzają zakłócenia do obrazu i osłabiają składowe wysokoczęstotliwościowe, co prowadzi do rozmycia krawędzi obiektów [69]. Jednym ze sposobów ograniczania utraty detali są moduły o zmiennym polu recepcyjnym, takie jak RepHELAN, które wykorzystują heterogeniczne konwolucje do zwiększania efektywnego pola recepcyjnego bez pogorszenia reprezentacji małych obiektów [72]. Z kolei w trudnych warunkach oświetleniowych stosuje się mechanizmy uwagi wieloskalowej, na przykład LSKA (*Large Separable Kernel Attention*), które mogą adaptacyjnie ograniczać wpływ lokalnych prześwieleń i koncentrować się na bardziej stabilnych zależnościach przestrzennych [62].

W rozległych scenach monitoringu, takich jak place miejskie, skrzyżowania czy parkingi, obiekty znajdujące się daleko od kamery klasyfikowane są jako małe jeżeli zajmują na obrazie mniej niż 32×32 , a często stanowią nawet mniej niż 20×20 pikseli [45]. W klasycznych sieciach konwolucyjnych kolejne operacje zmniejszania rozdzielczości, realizowane na przykład przez konwolucje ze skokiem

lub warstwy MaxPooling, mogą prowadzić do utraty informacji o tak małych obiektach jeszcze przed przekazaniem cech do głowic predykcyjnych [45]. Aby temu przeciwdziałać, nowoczesne architektury wykorzystują mechanizmy wzmacniające sygnał małych obiektów. Jednym z nich są gęste połączenia wspomagające, takie jak moduł SAF (*Superficial Assisted Fusion*) stosowany w części agregującej cechy (*neck*), który przekazuje do głębszych warstw informacje o wysokiej rozdzielczości przestrzennej z początkowych etapów ekstrakcji cech [72]. Innym rozwiązaniem jest regresja dystrybucyjna, na przykład DFL (*Distribution Focal Loss*) wykorzystywana w modelach YOLOv8 i YOLOv9, w której współrzędne ramki nie są modelowane jako pojedyncze wartości, lecz jako rozkłady prawdopodobieństwa wokół jej krawędzi. Pozwala to precyzyjniej opisać niepewność lokalizacji małego obiektu, także na poziomie subpikselowym [72].

2.3 Śledzenie wielu obiektów w pojedynczej kamerze

Śledzenie wielu obiektów (Multiple Object Tracking – MOT) w strumieniu wideo z pojedynczej kamery polega na jednoczesnym szacowaniu pozycji wielu poruszających się obiektów w czasie oraz na zachowaniu ich spójnej tożsamości w kolejnych klatkach sekwencji obrazów. Zadanie to wymaga rozwiązania trzech podstawowych problemów:

- **Asocjacji danych** (*ang. data association*) - powiązania niezależnych detekcji tych samych obiektów na przestrzeni kolejnych klatek wideo i przypisania im stałego, unikalnego identyfikatora *ang. (ang. track ID)* [6].
- **Inicjalizacji nowych ścieżek** — wykrywania momentu wejścia nowego, dotąd nieśledzonego obiektu w pole widzenia kamery i utworzenia dla niego nowej trajektorii [6].
- **Terminacji ścieżek** — identyfikacji momentu, w którym obiekt opuszcza monitorowaną scenę (lub zostaje długotrwale zasłonięty), w celu zakończenia aktualizacji jego trajektorii. Operacja ta zapobiega nieograniczonemu wzrostowi liczby śledzonych obiektów oraz redukuje błędy lokalizacji wynikające z długotrwałej predykcji ruchu [6].

2.3.1 Paradygmat śledzenia ścieżek (*ang. tracking-by-detection*)

Od około 2015 roku w dziedzinie wieloobiektowego śledzenia dominuje paradygmat śledzenia przez detekcję (*ang. tracking-by-detection lub detection-based tracking*) – DBT [70]. Podejście to opiera się na rozdzieleniu modułu lokalizacji przestrzennej od modułu odpowiedzialnego za utrzymanie tożsamości. Przepływ informacji realizowany jest kaskadowo: najpierw algorytm detekcyjny (np. głęboka sieć neuronowa z rodziny YOLO) niezależnie lokalizuje obiekty w każdej klatce wideo, a następnie moduł śledzący łączy te dyskretnie predykcje w ciągłe trajektorie [6, 70]. Taka separacja w architekturze systemu sprawia, że precyzja i czułość całego systemu MOT są bezpośrednio ograniczone przez jakość działania zastosowanego detektora bazowego [6].

2.3.2 Metody predykcji stanu i algorytmy skojarzenia

Aby efektywnie połączyć detekcje z klatki t z obiektami w klatce $t + 1$, nowoczesne systemy śledzenia wykorzystują kombinację filtrów estymacji stanu oraz algorytmów optymalizacji dyskretnej.

Filtr Kalmana

Do modelowania dynamiki ruchu obiektów stosuje się liniowy filtr Kalmana [30, 6]. Zakładając model ruchu o stałej prędkości na płaszczyźnie obrazu, filtr ten przewiduje przyszłą pozycję i wymiary ramki ograniczającej obiekt w kolejnej klatce wideo. Pozwala to zawęzić obszar poszukiwań i stabilizuje proces śledzenia nawet w przypadku chwilowych zakłóceń [6].

Algorytm węgierski

Powiązanie przewidzianych pozycji z nowymi detekcjami formułuje się jako problem skojarzenia w grafie dwudzielnym (*ang. bipartite matching*) o minimalnym koszcie [6]. Optymalne rozwiązanie tego zadania zapewnia algorytm węgierski [34, 6]. Metoda ta szuka optymalnych par w relacji jeden do jednego dla przewidzianych trajektorii i nowych detekcji, tak aby zminimalizować łączny koszt – np. odległość opartą na stopniu nakładania się ramek (IoU) [34, 6].

2.3.3 Ewolucja kluczowych algorytmów śledzenia

Rozwój algorytmów klasy MOT doprowadził do powstania kilku przełomowych architektur, stopniowo rozwiązujących problemy okluzji i wydajności obliczeniowej.

SORT (*ang. Simple Online and Realtime Tracking*), 2016:

Minimalistyczny algorytm łączący predykcję filtrem Kalmana i dopasowanie oparte na metryce IoU za pomocą algorytmu węgierskiego [70]. Dzięki prostej budowie cechuje się dużą szybkością – aktualizuje ścieżki z częstotliwością 260 Hz [6], jednak jest podatny na gubienie tożsamości w przypadku nawet krótkotrwałego zasłonięcia obiektu [6, 70].

Deep SORT, 2017

Rozwinięcie algorytmu SORT o mechanizm asocjacji oparty na głębokich cechach wizualnych [70]. Poprzez zastosowanie dedykowanej sieci CNN, wytrenowanej na zadanie reidentyfikacji (Re-ID), Deep SORT wyznacza deskryptory wizualne obiektów. Pozwala to na poprawne powiązanie tożsamości po długotrwałych okluzjach, drastycznie redukując liczbę błędnych przełączeń tożsamości (*ang. ID switches*) – w badaniach wykazano spadek tego błędu o 45% w stosunku do SORT [70].

ByteTrack, 2022

Autorzy tego algorytmu [73] zwrócili uwagę na wadę wcześniejszych metod, polegającą na odrzucaniu detekcji o niskim poziomie ufności - poniżej danego progu. Wskazali oni, że takie pomijane detekcje w rzeczywistości bardzo często reprezentują prawdziwe obiekty, które uległy częściowemu zasłonięciu lub rozmyciu w ruchu. Aby rozwiązać ten problem, wprowadzono dwustopniowy mechanizm asocjacji BYTE. W pierwszym kroku system standardowo dopasowuje detekcje o wysokiej ufności. Jeśli po tym etapie istnieją nieprzypisane trajektorie, algorytm przechodzi do drugiego kroku. Zamiast usuwać słabe detekcje jako szum tła, system sprawdza, czy ich geometria pokrywa się z miejscem, w którym filtr Kalmana przewidywał obecność śledzonego wcześniej obiektu. W tej fazie celowo wykorzystuje się wyłącznie podobieństwo przestrzenne (metrykę IoU), ignorując cechy wizualne, które przy silnej okluzji są niewiarygodne. Zaufanie słabej detekcji, o ile znajduje się ona w przewidywanym miejscu, pozwala z sukcesem kontynuować śledzenie trudnych do wykrycia obiektów, co ogranicza fragmentację ścieżek i redukuje liczbę błędów typu False Negative.

BoT-SORT, 2022

Zestaw optymalizacji (*ang. bag of tricks*) zintegrowany z algorytmem ByteTrack, który wprowadza trzy kluczowe innowacje rozwiązujące główne ograniczenia wcześniejszych metod [1]:

- **Kompensację ruchu kamery (CMC):** Wykorzystuje globalną kompensację ruchu (GMC) oraz rzadki przepływ optyczny do estymacji transformacji tła. Pozwala to na matematyczną korektę wektora stanu filtru Kalmana, zapobiegając rozbieżnościom i utracie ścieżek przy drganiach lub gwałtownych ruchach kamery.
- **Zmodyfikowany wektor stanu:** Wprowadza bezpośrednie modelowanie szerokości i wysokości ramki ograniczającej w filtrze Kalmana, rezygnując z powszechnie stosowanej miary proporcji boków. Zmiana ta poprawiła precyzję dopasowania predykcji do rzeczywistej szerokości obiektów.
- **Nowy mechanizm łączenia macierzy kosztów:** Integruje odległość przestrzenną (IoU) oraz unikalne cechy wyglądu (Re-ID). Zamiast stosowanej powszechnie sumy ważonej, algorytm jako ostateczny koszt przypisania wybiera wartość minimalną z obu macierzy (po uprzednim odfiltrowaniu niemożliwych par), co zwiększa skuteczność kojarzenia obiektów po okluzjach.

2.3.4 Ocena efektywności algorytmów MOT

Ilościowa ocena systemów MOT opiera się na ujednoliconych wskaźnikach matematycznych, które weryfikują zarówno zdolność detekcji, jak i stabilność asocjacji w czasie:

- **MOTA (Multiple Object Tracking Accuracy):** Ocenia ogólną dokładność systemu, sumując trzy typy błędów: pominięte obiekty (*False Negatives* – FN_t), fałszywe alarmy (*False Positives* – FP_t) oraz błędy przełączenia tożsamości (*ID Switches* – $IDSW_t$) [5, 43]. Wzór przyjmuje

postać:

$$\text{MOTA} = 1 - \frac{\sum_t (\text{FN}_t + \text{FP}_t + \text{IDSW}_t)}{\sum_t \text{GT}_t}$$

gdzie GT_t to rzeczywista liczba obiektów (*ang. Ground Truth*) w klatce t . Zakres tej metryki wynosi $(-\infty, 1]$ [43].

- **MOTP (Multiple Object Tracking Precision):** Mierzy geometryczną dokładność dopasowania ramek ograniczających dla poprawnie skojarzonych par obiektów [5, 43]:

$$\text{MOTP} = \frac{\sum_{i,t} d_{i,t}}{\sum_t c_t}$$

gdzie $d_{i,t}$ reprezentuje odległość (np. opartą na stopniu pokrycia IoU) dla dopasowanej pary i w klatce t , a c_t oznacza liczbę wszystkich poprawnych dopasowań w klatce t .

- **IDF1 (Identification F1 Score):** Ocenia długoterminową zdolność systemu do utrzymania stałych tożsamości, mierząc spójność na poziomie całych trajektorii [54]. Jest to średnia harmoniczna precyzji i czułości identyfikacji:

$$\text{IDF1} = \frac{2 \cdot \text{IDTP}}{2 \cdot \text{IDTP} + \text{IDFP} + \text{IDFN}}$$

gdzie IDTP, IDFP i IDFN reprezentują odpowiednio: poprawne, błędne oraz pominięte przypisania tożsamości, wyznaczone globalnie dla całego nagrania wideo [54].

2.3.5 Standardowe benchmarki (MOTChallenge)

Ocena porównawcza algorytmów śledzenia opiera się na standaryzowanej platformie testowej MOTChallenge:

- **MOT16 / MOT17:** Zestaw składający się z 14 sekwencji wideo reprezentujących zróżnicowane warunki środowiskowe (sceny miejskie, zmienne oświetlenie, ruch pieszy) [43]. Edycja MOT17 dostarcza wysoce precyzyjne adnotacje oraz dedykowany zestaw detekcji wygenerowanych przez trzy publiczne detektory (DPM, Faster R-CNN, SDP), co zapewnia uczciwość porównań metod [43, 73].
- **MOT20:** Zbiór danych zaprojektowany specjalnie dla scen o skrajnie wysokim zagęszczeniu tłumu (np. rynki miejskie, hale koncertowe), charakteryzujący się występowaniem gęstych skupisk obiektów generujących permanentne okluzje [15].

2.4 Systemy wielokamerowe - MTMC

2.4.1 Wprowadzenie do problematyki wielokamerowego śledzenia obiektów

Rozwój infrastruktury wizyjnej oraz skokowy wzrost mocy obliczeniowej systemów analitycznych doprowadziły do ewolucji systemów nadzoru wideo z podejścia lokalnego do w pełni rozproszonego.

Klasyczne algorytmy śledzenia w pojedynczej kamerze (*ang. Single-Camera Tracking – SCT*), choć osiągają obecnie zadowalającą dokładność dzięki wykorzystaniu zaawansowanych głębokich sieci neuronowych, napotykają fundamentalne ograniczenia przestrzenne. Pojedyncza kamera posiada ograniczone pole widzenia, w którym obiekty często ulegają długotrwałej okluzji (zasłonięciu), a jej fizyczne usytuowanie nierzadko narzuca trudną perspektywę obserwacji. Z tego powodu zainteresowanie badaczy i przemysłu skupia się obecnie na systemach śledzenia wielu obiektów w sieci kamer, powszechnie określanych w literaturze akronimami MTMC (*ang. Multi-Target Multi-Camera Tracking*) lub MCMT (*ang. Multi-Camera Multi-Target*) [54].

Głównym celem systemów MTMC jest łączenie danych z wielu strumieni wideo, pozwalając na przypisanie i utrzymanie unikalnego, globalnego identyfikatora (*ang. Global ID*) dla każdego śledzonego obiektu w całej sieci monitoringu [54, 46]. Systemy te stanowią fundament nowoczesnych, zautomatyzowanych przestrzeni handlowych, inteligentnych magazynów logistycznych oraz platform zarządzania ruchem miejskim. Pozwalają one na odtworzenie ciągłych, spójnych trajektorii obiektów w czasie i przestrzeni. Jest to niezbędne do analizy przepływu klientów, wczesnego wykrywania anomalii oraz automatyzacji i optymalizacji procesów przemysłowych [46].

Przestrzenie, w których wdraża się systemy MTMC, różnią się zakresem pokrycia wizualnego. Wyróżnia się dwa podstawowe sposoby rozmieszczenia kamer, które wpływają na dobór metod analizy [71]. Pierwszy obejmuje sieci kamer o pokrywających się polach widzenia (*ang. overlapping FOV*), dzięki którym ten sam obiekt może być obserwowany równocześnie z różnych perspektyw. Ułatwia to łączenie informacji geometrycznych. Drugi scenariusz, częstszy w praktyce i trudniejszy obliczeniowo, obejmuje układy z rozłącznymi polami widzenia (*ang. non-overlapping FOV*). W takim przypadku obiekt może zniknąć z obrazu jednej kamery, przejść przez martwą strefę (*ang. blind spot*), a następnie pojawić się w innej, oddalonej kamerze po czasie trudnym do oszacowania [54]. Oznacza to, że problem nie sprowadza się już tylko do śledzenia ruchu, lecz staje się zadaniem reidentyfikacji wymagającym modelowania zależności czasowo-przestrzennych [54].

2.4.2 Standardowa architektura potoku przetwarzania danych MTMC

Złożoność obliczeniowa i strukturalna śledzenia wielokamerowego wymaga zastosowania architektury zorientowanej na mikroserwisy oraz kaskadowe przetwarzanie potokowe (*ang. pipeline*). Współczesne rozwiązania przemysłowe i akademickie (m.in. platforma NVIDIA DeepStream SDK czy architektury z konkursów AI City Challenge) opierają się na powszechnie przyjętym paradygmacie śledzenia opartego na detekcji (*ang. tracking-by-detection*) [46, 12]. Paradygmat ten dzieli złożony problem na dwa główne, sekwencyjne etapy: lokalne śledzenie wewnątrz poszczególnych kamer oraz późniejszą, globalną asocjację między kamerami [71].

W typowym, dwuetapowym podejściu do MTMC, proces rozpoczyna się od wygenerowania krótkich ścieżek (*ang. tracklets*) dla każdego obiektu rejestrowanego niezależnie przez każdą z kamer [71, 54]. W pierwszym kroku moduły detekcji oparte na głębokich sieciach spłotowych (CNN) lokalizują sylwetki ludzi lub pojazdów na poszczególnych klatkach obrazu. Następnie lokalny

moduł śledzący (SCT), wykorzystując techniki estymacji ruchu (np. filtry Kalmana), łączy te niezależne predykcje w krótkoterminowe ścieżki wewnątrz pojedynczego widoku [6, 1]. Operacja ta ma na celu odfiltrowanie błędnych detekcji, uzupełnienie luk w trajektoriach powstałych na skutek krótkotrwałych okluzji oraz zagregowanie cech wizualnych. Pozwala to na wygenerowanie znacznie stabilniejszego i wiarygodniejszego opisu tożsamości przed przekazaniem danych do algorytmów reidentyfikacji [54, 12].

Wygenerowane w pierwszym kroku krótkie ścieżki (*ang. tracklets*) są następnie przekazywane do głównego etapu systemu MTMC, nazywanego globalną asocjacją między kamerami (*ang. Multi-Camera Association – MCA*) [71]. Zadaniem tego modułu jest powiązanie lokalnych ścieżek, pochodzących z różnych kamer i różnych przedziałów czasowych, z jedną spójną tożsamością globalną. Same informacje wizualne, takie jak deskryptory Re-ID, często nie wystarczają, ponieważ różne obiekty mogą wyglądać bardzo podobnie, na przykład pracownicy w jednakowych uniformach albo identyczne samochody na skrzyżowaniu. Dlatego ważną rolę odgrywają moduły, które sprawdzają zgodność skojarzeń w czasie i przestrzeni [12]. Moduły te odwzorowują pozycje obiektów na wspólną płaszczyznę sceny, a następnie wyznaczają takie skojarzenie w grafie, które daje najmniejszy koszt obliczony na podstawie odległości przestrzennej i podobieństwa wizualnego [54, 12].

Ponieważ system musi jednocześnie przetwarzać wiele strumieni wideo w czasie rzeczywistym lub z niewielkim opóźnieniem (*ang. near real-time*), jego architektura jest zwykle silnie równoległa. Zgodnie ze współczesnymi rozwiązaniami stosowanymi w praktyce kolejne klatki trafiają do modułów inferencyjnych korzystających z akceleracji sprzętowej [46]. Następnie uzyskane metadane są przesyłane przez rozproszone brokery komunikatów, takie jak Apache Kafka, do systemów bazodanowych przechowujących globalny graf tożsamości. Architektura mikroserwisowa ułatwia skalowanie całego systemu, na przykład z wykorzystaniem orkiestracji kontenerów w środowisku Kubernetes [46].

2.4.3 Problem rzutowania geometrycznego

Niezależne obrazy z poszczególnych kamer (we współrzędnych pikselowych u, v) należy sprowadzić do wspólnego układu odniesienia. Najczęściej stosowanym układem jest globalna metryczna płaszczyzna terenu (*ang. Bird's-Eye View – BEV*). W scenach o niskim zagęszczeniu standardową praktyką jest wyznaczenie dolnej krawędzi ramki ograniczającej jako fizycznego punktu styku obiektu z podłożem, a następnie rzutowanie tego punktu na globalną mapę za pomocą macierzy homografii lub modelu projekcyjnego kamery z wykorzystaniem jej parametrów zewnętrznych.

Niestety, podejście to zawodzi w przypadkach scen o wysokim zagęszczeniu tłumu lub przy złożonych okluzjach. Obiekty znajdujące się bliżej obiektywu zasłaniają dolne partie sylwetek obiektów znajdujących się w głębi sceny. Niemożliwe staje się więc precyzyjne wyznaczenie ich punktu styku z podłożem, a opieranie rzutowania na zniekształconej predykcji prowadzi do znacznych przesunięć wyestymowanej pozycji na globalnej płaszczyźnie BEV.

Rozwiązanie tego problemu w sieciach kamer o pokrywających się polach widzenia (*ang. overlapping FOV*) zostało zaprezentowane w 2008 roku jako koncepcja probabilistycznej mapy zajętości (*ang. Probabilistic Occupancy Map – POM*) [19].

2.4.4 Matematyczne sformułowanie algorytmu POM

Algorytm probabilistycznej mapy zajętości (*ang. Probabilistic Occupancy Map – POM*) służy do szacowania prawdopodobieństwa obecności osób na zdyskretyzowanej płaszczyźnie terenu (np. podłozie) na podstawie obrazów binarnych uzyskanych w procesie odejmowania tła, w sposób niejawni radząc sobie z problemem okluzji [19, 42]. W podejściu tym wykorzystano probabilistyczny model generatywny obrazu oraz przybliżenie wariacyjne pola średniego (*ang. mean-field approximation*), aby rozwiązać zadanie estymacji prawdopodobieństwa *a posteriori* $P(X | B)$, gdzie X opisuje wektor stanów mapy zajętości, a B zbiór obrazów binarnych ze wszystkich kamer [19].

Zmienne i założenia modelu Płaszczyzna terenu podlega dyskretyzacji na siatkę G komórek, z których każda reprezentuje potencjalną lokalizację osoby [19]. Dla każdej komórki $k \in \{1, \dots, G\}$ definiuje się binarną zmienną losową $X_k \in \{0, 1\}$, gdzie $X_k = 1$ oznacza obecność człowieka [19]. Całą mapę zajętości opisuje wektor stanów $X = (X_1, \dots, X_G)$. Głównym celem algorytmu jest estymacja prawdopodobieństwa *a posteriori* $P(X | B)$ względem obserwowanych obrazów binarnych $B = (B^1, \dots, B^C)$ pozyskanych z C kamer [19, 42].

W celu uproszczenia obliczeń model zakłada niezależność *a priori* pomiędzy komórkami, tzn. $P(X) = \prod_{k=1}^G P(X_k)$, oraz przyjmuje stałe, niezwykle rzadkie prawdopodobieństwo zajętości pojedynczej komórki, np. $\epsilon_k = P(X_k = 1) \ll 1$ [19].

Dla każdej kamery $c \in \{1, \dots, C\}$ wyznaczany jest obraz binarny B^c , uzyskany w procesie wyodrębniania tła (maska pierwszego planu). Algorytm POM buduje dla każdej z nich syntetyczny obraz A^c na podstawie aktualnej mapy zajętości X . W pozycji każdej komórki k , dla której $X_k = 1$, rzutowany jest prostokąt A_k^c przybliżający sylwetkę stojącej osoby, widzianej z perspektywy kamery c [19]. Formalnie, wygenerowany obraz syntetyczny jest wynikiem pikselowej sumy logicznej (*ang. union*) tych sylwetek:

$$A^c = \bigoplus_{k=1}^G X_k A_k^c$$

Przyjmuje się, że poszczególne widoki z kamer są warunkowo niezależne, a gęstość prawdopodobieństwa obrazu obserwowanego zależy wyłącznie od odpowiadającego mu obrazu syntetycznego [19]:

$$P(B | X) = \prod_{c=1}^C P(B^c | A^c) = \prod_{c=1}^C \frac{1}{Z} \exp(-\Psi(B^c, A^c))$$

gdzie Z to stała normalizacyjna, a $\Psi(\cdot, \cdot)$ jest miarą zdefiniowanej ad hoc „pseudoodległości” pomiędzy obrazami binarnymi.

Miarę $\Psi(B, A)$ zaprojektowano tak, aby nakładała karę za niezgodność pikseli między obrazem obserwowanym B a syntetycznym A :

$$\Psi(B, A) = \frac{1}{\sigma} \frac{|B \otimes (1 - A) + (1 - B) \otimes A|}{|A|}$$

gdzie $\otimes$ oznacza iloczyn pikselowy (część wspólną), $|A|$ sumę wartości pikseli obrazu A (pole powierzchni), a σ to parametr skali modelujący jakość algorytmu wyodrębniania tła [19]. Intuicyjnie, licznik wzoru zlicza liczbę błędnych pikseli (fałszywe alarmy i pominięcia względem obrazu syntetycznego), natomiast mianownik normalizuje tę wielkość przez pole powierzchni prostokąta, co zapobiega faworyzowaniu obiektów o większych rozmiarach w procesie optymalizacji [19].

Ponieważ bezpośrednie wyznaczenie dokładnego rozkładu *a posteriori* jest niemożliwe obliczeniowo ze względu na wykładniczo rosnącą przestrzeń kombinacji mapy zajętości, autorzy algorytmu POM zastosowali przybliżenie wariacyjne pola średniego (*ang. mean-field approximation*) [19]. Proces ten polega na poszukiwaniu uproszczonego rozkładu prawdopodobieństwa (zakładającego niezależność poszczególnych komórek siatki), który minimalizuje dywergencję Kullbacka-Leiblera względem rzeczywistego rozkładu [19]. Prowadzi to do układu równań punktu stałego, rozwiązywanego w sposób iteracyjny. W celu zapewnienia wysokiej wydajności, niezbędnej do przetwarzania sygnału wideo z wielu kamer, w praktycznej implementacji wykorzystano strukturę obrazów całkowych. Pozwala to na błyskawiczne aktualizowanie prawdopodobieństw obecności obiektów w kolejnych iteracjach, aż do osiągnięcia zbieżności modelu [19].

Interpretacja w kontekście wizji komputerowej Kluczową zaletą algorytmu POM jest jego zdolność do naturalnego i skutecznego radzenia sobie ze zjawiskiem okluzji. W modelu generatywnym obecność obiektu w jednej komórce „tłumaczy” brak widocznych pikseli pierwszego planu dla potencjalnego obiektu znajdującego się tuż za nim, wzdłuż tej samej osi projekcji kamery [19]. Pozwala to modelowi na poprawne rozróżnianie złożonych konfiguracji osób, zarówno w sytuacjach silnych skrótów perspektywicznych, jak i w przypadku częściowej utraty informacji z niektórych kamer [19]. Ponadto, wykorzystanie algorytmu POM jako początkowego etapu detekcji na mapie zajętości zapewnia wysoką precyzję lokalizacji obiektów we współrzędnych świata [42]. Stanowi to podstawę stabilnego śledzenia globalnych trajektorii w czasie [42].

2.5 Reidentyfikacja obiektów - Re-ID

2.5.1 Definicja i sformułowanie problemu reidentyfikacji

Reidentyfikacja obiektów (*ang. person/vehicle re-identification* – Re-ID) umożliwia kojarzenie tożsamości celów ruchomych przemieszczających się pomiędzy fizycznie rozłącznymi obszarami, które są obserwowane przez kamery o niepokrywających się polach widzenia (*ang. non-overlapping fields of view*) [54, 77].

Proces ten definiuje się jako zadanie wyszukiwania informacji wizualnej (*ang. image retrieval*) w relacji 1 : N [77, 78]. Dane wejściowe dla algorytmu Re-ID składają się z dwóch podstawowych zbiorów:

- **Zapytania** (*ang. query* lub *probe*) – reprezentowanego przez wycięty kadr obrazu zawierający sylwetkę poszukiwanego obiektu (człowieka lub pojazdu). Kadr ten jest najczęściej generowany przez moduł detekcji lub śledzenia wewnątrz pojedynczej kamery [54, 77].
- **Galerii** (*ang. gallery*) – wielkoskalowej bazy danych (N elementów) zawierającej kadry obiektów referencyjnych zarejestrowanych przez inne kamery w sieci, wśród których algorytm poszukuje dopasowań [77]. W warunkach rzeczywistych galeria ta jest często generowana automatycznie przez detektory wizyjne, co nieuniknienie wprowadza do niej szum w postaci fałszywych detekcji [78].

Skuteczne rozwiązanie tego problemu opiera się na dwóch podstawowych elementach [38]. Pierwszym jest wyznaczenie możliwie najlepiej rozróżniającej reprezentacji cech dla zapytania i obrazów z galerii. Drugim jest nauka miary podobieństwa, czyli określenie, jak porównywać obiekty w przestrzeni cech, aby uporządkować obrazy z galerii od najbardziej do najmniej podobnych względem zapytania [38, 77].

Głównym wyzwaniem jest to, że model musi opierać się wyłącznie na cechach wyglądu zewnętrznego, takich jak kolor i struktura odzieży czy ogólna sylwetka [51]. Wynika to z faktu, że niska rozdzielczość oraz skośne ustawienie kamer utrudniają wydobycie precyzyjnych cech biometrycznych, takich jak geometria twarzy czy tekstura tęczówki [54, 78]. Dodatkowym utrudnieniem są duże różnice między obrazami tej samej osoby, spowodowane zmianami oświetlenia, perspektywy widzenia kamery, pozycji obiektu oraz częstymi okluzjami [38, 78].

2.5.2 Klasyczne podejścia oparte na cechach projektowanych ręcznie

Początkowo badania nad reidentyfikacją osób koncentrowały się na ręcznym projektowaniu deskryptorów wizualnych (*ang. hand-crafted features*) odpornych na transformacje geometryczne i fotometryczne sceny oraz na optymalizacji klasycznych miar odległości (*ang. metric learning*) [77]. Do najczęściej cytowanych rozwiązań tej rodziny należą:

- **Histogramy koloru**: Reprezentujące globalny lub lokalny rozkład barw w różnych przestrzeniach (RGB, HSV, YUV). Metody te charakteryzowały się prostotą obliczeniową, lecz wykazywały skrajną wrażliwość na zmiany oświetlenia i różnice w temperaturze barwowej i balansie bieli kamer.
- **SDALF** (*ang. Symmetry-Driven Accumulation of Local Features*): Algorytm opiera się na lokalizacji osi symetrii i asymetrii ludzkiego ciała, co pozwala na podział sylwetki na stabilne regiony (głowa, tułów, nogi). Cechy lokalne (takie jak tekstury oparte na filtrach Gabora i histogramy kolorów) są akumulowane wzdłuż tych osi, co pozwala na uzyskanie reprezentacji odpornej na rotację obiektu i zmiany pozycji [17].

- **LOMO** (*ang. Local Maximal Occurrence*): Zaawansowana metoda opracowana przez Liao i in. [38]. Aby uodpornić system na wahania jasności, obrazy przetwarzane są transformatą Retinexa, która symuluje percepcję ludzkiego oka, uwydatniając detale w cieniach i stabilizując barwy [38]. Do ekstrakcji tekstur wykorzystano niezależny od skali intensywności i wysoce odporny na szum operator SILTP [38]. Główną innowacją LOMO jest jednak proces analizy poziomych pasów obrazu za pomocą przesuwających się okien. Algorytm rejestruje maksymalną wartość wystąpienia danej cechy (np. wzoru lub koloru) wzdłuż całej linii poziomej [38]. Taka agregacja sprawia, że ostateczny wektor cech jest niezwykle odporny na rotację obiektu (zmiany punktu widzenia kamery – *ang. viewpoint invariance*), eliminując potrzebę dokładnego lokalizowania poszczególnych partii ciała [38].

2.5.3 Uczenie reprezentacji metrycznych i sieci syjamskie

Zastosowanie głębokich sieci neuronowych do reidentyfikacji pozwoliło na zjednoczenie etapu ekstrakcji cech. Najważniejszą rolę w tym procesie odgrywa głębokie uczenie metryk (*ang. deep metric learning*) w podejściu *end-to-end*, dodatkowe, osobne etapy klasycznego uczenia odległości stały się zbędne [27].

Wczesne architektury głębokie bazowały na sieciach syjamskich (*ang. siamese networks*) składających się z dwóch lub więcej bliźniaczych gałęzi o współdzielonych wagach, do których wprowadzano pary obrazów [78]. Sieci te optymalizowano przy użyciu funkcji straty kontrastowej (*ang. contrastive loss*), która dąży do minimalizacji odległości między cechami par obrazów o tej samej tożsamości (pozytywnych) oraz maksymalizacji odległości par o różnych tożsamościach (negatywnych), tak aby oddalić je od siebie powyżej zdefiniowanego marginesu α [78].

Powszechnie stosowaną alternatywą stała się funkcja straty trójkowej (*ang. triplet loss*), która operuje na zestawach złożonych z próbki kotwiczącej (x_a), próbki pozytywnej (x_p) oraz próbki negatywnej (x_n) [27]. Klasyczny triplet loss dąży do spełnienia warunku:

$$D(x_a, x_p) + m < D(x_a, x_n) \quad (19)$$

gdzie $D(\cdot, \cdot)$ oznacza euklidesową odległość w przestrzeni cech, a m reprezentuje narzucony margines bezpieczeństwa [27].

Hermans i in. [27] wykazali, że efektywność tej straty w Re-ID ulega drastycznej poprawie przy zastosowaniu strategii próbkowania najtrudniejszych przykładów wewnątrz mini-batcha (*ang. batch-hard triplet loss*):

$$\mathcal{L}_{BH} = \sum_{i=1}^P \sum_{a=1}^K \left[m + \max_{p=1 \dots K} D(x_i^a, x_i^p) - \min_{\substack{j=1 \dots P \\ j \neq i \\ n=1 \dots K}} D(x_i^a, x_j^n) \right]_+ \quad (20)$$

gdzie P oznacza liczbę unikalnych tożsamości w batchu, K to liczba obrazów na każdą tożsamość, x_i^a oznacza a -tą próbkę kotwiczącą należącą do i -tej tożsamości, a $[\cdot]_+$ reprezentuje funkcję $\max(0, \cdot)$ [27]. Algorytm ten koncentruje proces optymalizacji wyłącznie na najmniej podobnych parach pozytywnych oraz najbardziej zbliżonych parach negatywnych, eliminując dominację łatwych przykładów [27].

2.5.4 Architektury splotowe (CNN) dedykowane do zadań Re-ID

Większość splotowych modeli reidentyfikacji osób wykorzystuje architekturę ResNet-50 jako sieć bazową (*ang. backbone*) ze względu na korzystny kompromis między dokładnością a kosztami obliczeniowymi [54, 77]. Ze względu na specyfikę zadania opracowano dedykowane paradygmaty uczenia reprezentacji:

- **IDE** (*ang. ID-discriminative Embedding*): Model wprowadzony przez Zhenga i in. [78]. Traktuje on proces treningowy jako problem klasyfikacji wieloklasowej, w którym każda tożsamość w zbiorze uczącym stanowi odrębną klasę - model optymalizowany jest z użyciem funkcji straty opartej na entropii krzyżowej [78]. Podczas wnioskowania odrzuca się warstwę klasyfikacyjną, a wyjściowy wektor cech z przedostatniej warstwy (GAP) służy bezpośrednio jako deskryptor tożsamości obiektu [78].
- **PCB** (*ang. Part-based Convolutional Baseline*): Model zaproponowany przez Suna i in. [59] w celu rozwiązania problemu utraty informacji przestrzennych w globalnych deskryptorach. PCB rezygnuje z globalnego uśredniania cech (GAP) na rzecz równomiernego, sztywnego podziału wyjściowej mapy cech na p poziomych pasów, najczęściej $p = 6$ [59]. W przeciwieństwie do wcześniejszych metod, podział ten ma charakter wyłącznie przestrzenny i nie wymaga dopasowania do semantycznych części ciała. Dla każdego pasa obliczany jest niezależny lokalny wektor cech, optymalizowany osobną funkcją straty, co pozwala modelowi uchwycić drobne, lokalne szczegóły wyglądu (*fine-grained features*) [59].

2.5.5 Architektury oparte na Transformerach (SOTA)

Obecnie do najskuteczniejszych rozwiązań w dziedzinie Re-ID należą modele oparte na architekturze Vision Transformer (ViT). Dzięki mechanizmowi samoatencji (*ang. self-attention*) potrafią one modelować zależności długiego zasięgu (*ang. long-range dependencies*) bezpośrednio na podstawie sekwencji fragmentów obrazu (*ang. patches*), bez utraty szczegółów wynikającej z operacji zmniejszania rozdzielczości (*ang. downsampling*) charakterystycznych dla klasycznych sieci splotowych [25]. Przełomową architekturą tej grupy jest model **TransReID** opracowany przez He i in. [25]. Wprowadza on do struktury transformera dwa istotne moduły:

- **JPM** (*ang. Jigsaw Patch Module*): Moduł ten dzieli wektory cech fragmentów obrazu na kilka grup, dokonując na nich operacji przesunięcia (*ang. shift*) i przemieszania (*ang. patch shuffle*) przed przekazaniem ich do kolejnych warstw sieci [25]. Wymusza to na modelu uwzględnianie

szerszego kontekstu globalnego i ogranicza jego nadmierne skupienie na jednym dominującym fragmencie sylwetki. W rezultacie zwiększa się odporność systemu na częściowe okluzje oraz zmiany pozy śledzonego obiektu [25].

- **SIE** (*ang. Side Information Embeddings*): Pozwala na bezpośrednią integrację niewizualnych informacji kontekstowych, takich jak identyfikator kamery czy punkt widzenia, za pomocą wyuczalnych reprezentacji (*ang. learnable embeddings*) [25]. Modyfikuje to wejściową sekwencję Z_0 zawierającą już informacje o położenie zgodnie ze wzorem:

$$Z'_0 = Z_0 + \lambda S_{(C,V)} \quad (21)$$

gdzie $S_{(C,V)}$ oznacza połączoną reprezentację dla danej kamery i perspektywy, natomiast λ jest parametrem wagowym. Mechanizm ten pozwala ograniczyć błędy w przestrzeni cech wynikające ze specyfiki fizycznego położenia danej kamery, a tym samym zmniejszyć różnice w podobieństwie obiektów rejestrowanych z różnych ujęć [25].

2.5.6 Problem rozbieżności dziedzin i adaptacja bez nadzoru (UDA)

Jednym z najważniejszych ograniczeń praktycznych systemów Re-ID jest rozbieżność między środowiskiem treningowym a warunkami pracy systemu, czyli tzw. przesunięcie dziedzinowe (*ang. domain shift / domain gap*) [54]. Model, który działa bardzo dobrze na danych z jednego zbioru kamer, po przeniesieniu do innej sieci monitoringu często wyraźnie traci skuteczność [54, 20]. Wynika to z różnic w obrazie rejestrowanym przez różne kamery: inne jest oświetlenie, tło, perspektywa, jakość kompresji oraz sam wygląd osób widzianych z różnych punktów obserwacji [54, 20].

W odróżnieniu od klasycznych problemów nadzorowanego uczenia, w praktycznych zastosowaniach zwykle nie ma możliwości ręcznego opisanie dużej liczby danych z nowego środowiska. Z tego powodu rozwijane są metody nienadzorowanej adaptacji dziedzinowej (*ang. Unsupervised Domain Adaptation* – UDA) [20]. Ich celem jest przeniesienie wiedzy z domeny źródłowej do docelowej bez użycia etykiet dla danych docelowych. W literaturze Re-ID szczególnie ważne stały się podejścia oparte na schemacie nauczyciel–uczeń, ponieważ pozwalają wykorzystywać nieopisane dane z nowego środowiska w sposób kontrolowany i stosunkowo stabilny.

Jednym z najbardziej znanych rozwiązań tego typu jest algorytm **MMT** (*ang. Mutual Mean-Teacher*) zaproponowany przez Ge i in. [20]. Metoda ta wykorzystuje dwie równoległe uczone sieci oraz ich wersje nauczycielskie, których parametry nie są aktualizowane przez propagację błędu, lecz jako wykładnicza średnia krocząca (*ang. Exponential Moving Average* – EMA) wag modeli uczniów [20]. Dzięki temu model nauczyciela dostarcza bardziej stabilnych pseudoetykiet dla danych docelowych.

Kluczowym elementem MMT jest wzajemne uczenie się modeli na podstawie miękkich pseudoetykiet (*ang. soft pseudo-labels*) generowanych dla danych z domeny docelowej [20]. Zamiast przypisywać próbkę do jednej, sztywnej klasy, model otrzymuje rozkład prawdopodobieństwa, który lepiej oddaje niepewność predykcji. Takie podejście zmniejsza ryzyko błędów wynikających

ze zbyt pewnych, ale niepoprawnych przypisań i ogranicza narastanie szumu w kolejnych iteracjach uczenia [20].

W kontekście Re-ID jest to szczególnie istotne, ponieważ ta sama osoba może wyglądać bardzo różnie w zależności od kamery, kierunku patrzenia, odległości od obiektywu czy warunków oświetleniowych. Dodatkowo w obrazie często pojawia się tło, które nie niesie informacji o tożsamości, a jedynie utrudnia dopasowanie. Z tego powodu metody UDA dla Re-ID zwykle starają się nie tylko ograniczyć różnice między domenami, ale też wydobyć cechy najbardziej związane z samą osobą, a nie z otoczeniem.

2.5.7 Specyfika reidentyfikacji pojazdów w odniesieniu do osób

Choć reidentyfikacja pojazdów (*ang. vehicle Re-ID*) i reidentyfikacja osób opierają się na podobnych podstawach, oba zadania różnią się pod względem cech obiektu i trudności praktycznych:

- Sylwetka człowieka może zmieniać się w czasie, na przykład w zależności od pozycji ciała, natomiast pojazd ma zwykle stały kształt.
- Samochody produkowane seryjnie często wyglądają bardzo podobnie, nawet jeśli są to różne egzemplarze tego samego modelu. Z tego powodu sieć musi szukać drobnych, wyróżniających cech, takich jak tablice rejestracyjne, naklejki, uszkodzenia karoserii czy przedmioty widoczne we wnętrzu pojazdu.
- Wygląd pojazdu zależy od punktu widzenia kamery. Przód, bok i tył auta mogą wyglądać zupełnie inaczej, dlatego w praktyce często stosuje się podejście wielowidokowe (*ang. multi-view Re-ID*).

Do oceny algorytmów Re-ID pojazdów stosuje się dedykowane zbiory danych, z których najważniejsze to **VeRi-776** oraz **VehicleID**. Pierwszy z nich zawiera blisko 50 000 zdjęć 776 pojazdów zarejestrowanych przez 20 kamer, a drugi ponad 220 000 obrazów. Zbiory te stanowią obecnie podstawowy punkt odniesienia przy testowaniu metod w trudnych warunkach ruchu drogowego [25].

2.5.8 Prywatność i alternatywne metody identyfikacji

Tradycyjne podejścia do Re-ID, oparte na pełnych obrazach RGB sylwetek ludzi, budzą uzasadnione obawy związane z ochroną prywatności, zwłaszcza w kontekście przepisów RODO. Z tego powodu coraz większe zainteresowanie budzą metody, które ograniczają ilość przetwarzanych danych już na etapie projektowania systemu (*ang. privacy by design*):

- **Silhouette Re-ID**: Podejście polegające na identyfikacji osób wyłącznie na podstawie binarnych masek sylwetek wyodrębnionych z obrazu, na przykład za pomocą algorytmów BGS. Metody te pomijają teksturę ubrań i cechy twarzy, a zamiast tego opierają się na kształcie sylwetki i dynamice chodu (*ang. gait-based Re-ID*) [9]. Powszechnie stosowany model **GaitSet** traktuje sekwencję chodu jako zbiór sylwetek, dzięki czemu dobrze radzi sobie z brakującymi klatkami i zmianami tempa poruszania się [9].

- **Skeleton-based Re-ID**: Podejście wykorzystujące estymację pozy, na przykład za pomocą OpenPose, do wyznaczenia współrzędnych kluczowych punktów ciała człowieka [51]. Modele takie jak **TranSG** opisują ruch człowieka za pomocą grafu szkieletowego i uczą się cech ruchu bez korzystania z obrazu twarzy czy odzieży. Takie rozwiązanie zapewnia lepszą ochronę prywatności, a jednocześnie pozwala zachować dobrą skuteczność rozpoznawania między kamerami [51].

2.6 Rzutowanie na plan terenu (Bird's-Eye View - BEV) i geometria sceny

Widok z lotu ptaka (*ang. Bird's-Eye View – BEV*) polega na rzutowaniu sceny na jedną, wspólną płaszczyznę odniesienia, najczęściej na fizyczną płaszczyznę terenu lub jezdni. Reprezentacja BEV pozwala na określenie położenia obiektów w metrycznym układzie współrzędnych, co całkowicie uniezależnia analizę od zniekształceń perspektywicznych specyficznych dla pojedynczej kamery [19, 36].

Współcześnie, w systemach autonomicznej jazdy, BEV stanowi standardową przestrzeń reprezentacji pośredniej (*ang. intermediate representation*). Architektury percepcyjne przekształcają surowe dane z kamer lub sensorów LiDAR do widoku z lotu ptaka, a następnie dedykowane moduły realizują w tej zunifikowanej przestrzeni zadania takie jak detekcja obiektów 3D, semantyczna segmentacja mapy oraz predykcja trajektorii ruchu [36, 75].

W kontekście systemów monitoringu wizyjnego (CCTV), rzutowanie na płaszczyznę BEV pełni analogiczną, fundamentalną rolę. Umożliwia ono geometryczną fuzję informacji z wielu kamer w ramach jednego, globalnego układu współrzędnych, co stanowi podstawę do precyzyjnej analizy i śledzenia globalnych trajektorii obiektów w czasie na planie terenu [19].

2.6.1 Geometryczne rzutowanie na plan terenu i Inverse Perspective Mapping

Klasycznym sposobem uzyskania widoku z lotu ptaka jest rzutowanie obrazu perspektywicznego na płaszczyznę terenu za pomocą macierzy homografii. W podejściu tym zakłada się, że podłoże jest idealnie płaskie, a parametry kamery (macierz wewnętrzna oraz macierz rotacji i wektor translacji) są znane dzięki procesowi kalibracji [24, 61]. Zgodnie z modelem kamery otworkowej, punkt w przestrzeni trójwymiarowej rzutowany jest na płaszczyznę obrazu przy użyciu macierzy projekcji $P = K[R \mid t]$. Zakładając, że rozpatrywane punkty leżą bezpośrednio na płaszczyźnie gruntu (wysokość $Z = 0$), przekształcenie to redukuje się do macierzy homografii H , która jednoznacznie odwzorowuje współrzędne pikselowe obrazu (u, v) na metryczne współrzędne w układzie planu terenu (x, y) [24, 61].

Technika odwrotnego mapowania perspektywicznego (*ang. Inverse Perspective Mapping – IPM*) polega na zastosowaniu tak wyznaczonej homografii do wszystkich pikseli obrazu wejściowego [36]. W rezultacie uzyskuje się syntetyczny obraz z lotu ptaka, na którym elementy infrastruktury

drogowej (np. linie jezdni, pasy ruchu) oraz obiekty prezentowane są z pominięciem zniekształceń perspektywicznych [28]. IPM jest powszechnie stosowana w zadaniach związanych z detekcją pasów i oznakowania drogowego. Jej główną zaletą jest prostota obliczeniowa oraz bezpośrednie oparcie na fizycznym modelu geometrycznym kamery. Fundamentalnym ograniczeniem pozostaje jednak wysoka wrażliwość na błędy kalibracji (np. drgania pojazdu zmieniające pochylenie kamery) oraz fakt, że założenie o idealnie płaskim podłożu (*ang. flat-ground assumption*) rzadko bywa w pełni spełnione w rzeczywistych warunkach drogowych [28, 36].

2.6.2 Siatka BEV i probabilistyczne mapy zajętości

W praktycznych systemach widok z lotu ptaka jest zwykle reprezentowany jako siatka komórek (*ang. BEV grid*). Płaszczyzna terenu dzielona jest na prostokątne pola o zadanym rozmiarze (np. 10–50 cm), z których każda opisuje określony fragment przestrzeni [42]. Klasycznym podejściem, wywodzącym się z robotyki mobilnej, jest tworzenie siatek zajętości (*ang. occupancy grid mapping*). Dla każdej komórki siatki szacuje się prawdopodobieństwo, że jest ona zajęta przez przeszkodę lub obiekt. Informacje z różnorodnych czujników (kamery, LiDAR, radar) łączone są w ramach probabilistycznego modelu, a mapa zajętości jest na bieżąco aktualizowana w czasie [42].

Probabilistyczna mapa zajętości (*ang. Probabilistic Occupancy Map – POM*) zaproponowana przez Fleureta i in. jest w istocie szczególnym przypadkiem siatki zajętości na płaszczyźnie terenu [42]. Autorzy dyskretyzują plan sceny na siatkę komórek i estymują prawdopodobieństwo obecności osoby w każdym z tych pól na podstawie obrazów binarnych uzyskanych w procesie wyodrębniania tła z wielu kamer [19]. W niniejszej pracy siatka BEV jest bezpośrednio utożsamiana z siatką POM: dla każdej komórki $\mathcal{G}_k$ przechowywana jest wartość brzegowego prawdopodobieństwa $P(X_k = 1)$, a wynikowy widok z lotu ptaka przedstawia globalny rozkład zajętości obiektów na planie terenu w danej chwili czasowej.

2.6.3 Widok z lotu ptaka w systemach autonomicznej jazdy

W badaniach nad autonomiczną jazdą widok z lotu ptaka (BEV) stał się jedną z kluczowych przestrzeni reprezentacji pośredniej. Sieci neuronowe realizują transformację surowych danych z kamer i sensorów LiDAR do przestrzeni BEV, a zadania takie jak detekcja trójwymiarowa, semantyczna segmentacja mapy oraz predykcja ruchu pojazdów definiowane są bezpośrednio w tym układzie współrzędnych [36, 75]. Najnowsze przeglądy literaturowe podkreślają kilka fundamentalnych zalet BEV: przejrzyste odwzorowanie sceny w układzie metrycznym, ułatwienie fuzji danych z wielu różnorodnych sensorów, możliwość prostego definiowania stref na drodze (np. pasów, skrzyżowań, obszarów wyłączonych z ruchu) oraz intuicyjne wykorzystanie tak zagregowanych informacji przez moduły planowania ruchu i sterowania pojazdem [36, 75].

Te same cechy odgrywają istotną rolę w wielokamerowych systemach monitoringu wizyjnego (CCTV). Widok z lotu ptaka pozwala na mapowanie położenia pieszych i pojazdów na punkty odpowiadające rzeczywistym lokalizacjom w przestrzeni fizycznej, całkowicie niezależniając analizę od

perspektywy narzucanej przez poszczególne kamery. Dzięki temu możliwa staje się analiza globalnego ruchu na planie całej sceny, a nie tylko w obrębie odizolowanych obrazów dwuwymiarowych [19].

2.6.4 Metody BEV oparte na głębokim uczeniu

Nowoczesne metody generowania widoku z lotu ptaka odchodzą od bezpośredniego polegania na sztywnej geometrii na rzecz modeli opartych na głębokich sieciach neuronowych. Uczą się one transformacji z widoku perspektywicznego do przestrzeni BEV w sposób czysto oparty na danych, bez konieczności ręcznego definiowania wszystkich kroków rzutowania [36].

Przełomowym przykładem metody opartej na estymacji głębi (podejście 2D-3D) jest architektura **Lift, Splat, Shoot (LSS)** [49]. W fazie *Lift*, model estymuje dyskretny rozkład głębi dla każdego piksela obrazu, a następnie „podnosi” dwuwymiarowe cechy do postaci trójwymiarowego ostrosłupa widzenia kamery (*ang. viewing frustum*) [75]. Następnie, w fazie *Splat*, cechy te są rzutowane i agregowane na płaską siatkę BEV. Z kolei faza *Shoot* polega na wyznaczeniu z tej reprezentacji mapy kosztów (*ang. cost map*), służącej do semantycznej segmentacji sceny oraz planowania bezkolizyjnych trajektorii ruchu pojazdu [49, 75].

Alternatywnym, wysoce skutecznym podejściem jest model **BEVFormer**. Wykorzystuje on architekturę Transformerów do bezpośredniego generowania reprezentacji BEV z sekwencji obrazów wielokamerowych [37]. Model ten wprowadza siatkę wyuczalnych wektorów zapytań (*ang. grid-shaped BEV queries*), z których każdy odpowiada konkretnej komórce w przestrzeni BEV. Stosując zaawansowane mechanizmy uwagi przestrzennej (*ang. spatial cross-attention*) oraz czasowej (*ang. temporal self-attention*), sieć elastycznie łączy i filtruje informacje z różnych ujęć kamer oraz wcześniejszych klatek wideo [37].

W przeciwieństwie do klasycznej metody IPM, rozwiązania oparte na głębokim uczeniu wykazują znacznie mniejszą wrażliwość na błędy kalibracji czujników i lepiej radzą sobie w scenach, w których założenie o idealnie płaskim podłożu nie jest spełnione [36]. Ich główną wadą pozostaje jednak konieczność posiadania potężnych zbiorów danych z dokładnymi adnotacjami w przestrzeni 3D lub BEV, a także charakterystyczny dla modeli głębokich brak pełnej przejrzystości i interpretowalności procesu decyzyjnego [36, 75].

2.6.5 Zastosowanie widoku z góry w systemach wielokamerowych MTMC

W systemach śledzenia wielu obiektów w sieci kamer (MTMC), widok z lotu ptaka (BEV) stanowi naturalną, wspólną przestrzeń reprezentacji. Rzutując detekcje z poszczególnych kamer na plan terenu, trajektorie obiektów można przedstawić jako sekwencje punktów w jednej, globalnej i metrycznej siatce. Ułatwia to późniejsze kojarzenie ścieżek między kamerami oraz modelowanie prawdopodobieństwa przejść [19]. Probabilistyczna mapa zajętości (POM) dostarcza w każdej chwili precyzyjnych informacji o tym, które komórki siatki są zajęte. Na tej podstawie algorytmy mogą definiować strefy wejścia i wyjścia, analizować główne kierunki ruchu, a także prognozować czas pojawienia się obiektu w polu widzenia kolejnej kamery [19].

Jak zostanie zaprezentowane w dalszej części pracy, docelowy system MTMC wykorzystuje widok z lotu ptaka jako absolutny fundament fuzji danych. Moduły śledzenia lokalnego (SCT) generują dla każdej kamery niezależne trajektorie w przestrzeni obrazu dwuwymiarowego, które następnie są transformowane do układu BEV. Z kolei moduł globalnej asocjacji kamer (MCA) łączy ze sobą cechy wizualne (deskryptory Re-ID) oraz informacje czasowo-przestrzenne w zunifikowanej przestrzeni z lotu ptaka, aby skutecznie utrzymać spójne identyfikatory obiektów w całej monitorowanej sieci monitoringu [71, 12].

Rozdział 3

Przegląd istniejących rozwiązań

W klasycznym ujęciu, architektura potoku przetwarzania danych MTMC dzieli się na dwa główne etapy: lokalne śledzenie wewnątrz poszczególnych kamer (ang. Single-Camera Tracking – SCT), generujące krótkie trajektorie (ang. tracklets), oraz późniejszą, globalną asocjację między kamerami (ang. Multi-Camera Association – MCA), w której informacje wizualne i czasowo-przestrzenne są integrowane w celu nadania unikalnych identyfikatorów. Wdrożenie tak zaawansowanego potoku na urządzeniach klasy brzegowej lub stacjach roboczych o ograniczonej charakterystyce energetycznej wymusza specyficzne, rygorystyczne podejście do doboru technologii oprogramowania. Niniejszy rozdział stanowi przegląd istniejących rozwiązań algorytmicznych oraz architektonicznych, a także uzasadnienie wyboru stosu technologicznego dla systemu MTMC budowanego w środowisku macOS. Docelową platformą uruchomieniową jest komputer MacBook Pro wyposażony w układ Apple Silicon M5. Z uwagi na unikalną specyfikę architektury sprzętowej Apple, standardowe i powszechnie stosowane w przemyśle rozwiązania oparte na środowisku NVIDIA CUDA muszą zostać zastąpione alternatywnym stosem optymalizacyjnym, gwarantującym najwyższą wydajność przy wykorzystaniu akceleratorów wbudowanych w układ M5. Dokonano tu szczegółowej ewaluacji dostępnych języków programowania, modeli detekcji z rodziny YOLO, głębokich architektur transformatorowych dostępnych za pośrednictwem platformy Hugging Face oraz klasycznych algorytmów wizji komputerowej w ramach biblioteki OpenCV.

3.1 Specyfika sprzętu oraz środowiska programistycznego

3.1.1 Architektura sprzętowa: Apple Silicon M5 a akceleracja obliczeń tensorowych

Głównym czynnikiem determinującym architekturę opisywanego systemu jest zastosowanie układu SoC (ang. *System-on-a-Chip*) z rodziny Apple Silicon M5. Architektura ta wprowadza zmianę w sposobie zarządzania pamięcią operacyjną w porównaniu do klasycznych środowisk opartych na architekturze x86 i dedykowanych kartach graficznych, co bezpośrednio wpływa na wydajność przetwarzania modeli głębokiego uczenia [18].

Układy z serii M wykorzystują zunifikowaną architekturę pamięci (*ang. Unified Memory Architecture – UMA*). Oznacza to, że procesor centralny (CPU), wielordzeniowy procesor graficzny (GPU) oraz sprzętowy akcelerator macierzowy (*ang. Neural Engine*) współdzielą tę samą, wysoce przepustową pamięć RAM [18, 57]. W klasycznych systemach wielokamerowych, opartych np. na systemie Ubuntu i kartach z serii NVIDIA RTX, konieczność ciągłego przesyłania zdekodowanych klatek obrazu z pamięci operacyjnej do pamięci karty graficznej (VRAM) przez magistralę PCIe stanowi istotne ograniczenie wydajności potoku przetwarzania (*ang. pipeline*) [57].

Architektura UMA całkowicie eliminuje narzut czasu związany z kopiowaniem tych danych. Układy obliczeniowe uzyskują bezpośredni dostęp do surowych klatek wideo natychmiast po ich zdekodowaniu przez procesor główny [57]. Zmniejszenie tych opóźnień znacznie przyspiesza zarówno procesy wstępnego przetwarzania obrazu, jak i samo wnioskowanie (*ang. inference*) na modelach wizyjnych [57].

Układy Apple Silicon posiadają również sprzętowy akcelerator (*ang. Neural Engine*), zoptymalizowany pod kątem operacji mnożenia macierzy (*ang. Matrix Multiplication – MatMul*), które stanowią fundament obliczeniowy głębokich sieci neuronowych [57]. Aby w pełni wykorzystać ten potencjał, firma Apple opracowała interfejs programistyczny (*ang. Metal Performance Shaders – MPS*) [18]. Działa on jako niskopoziomowa alternatywa dla przemysłowego standardu NVIDIA CUDA, bezpośrednio mapując operacje uczenia maszynowego na architekturę układów z serii M [18, 2].

Należy jednak zaznaczyć, że wsparcie popularnych frameworków (m.in. *PyTorch*) dla środowiska MPS wciąż znajduje się w fazie rozwoju i posiada pewne braki optymalizacyjne [18, 2]. Brak natywnej implementacji specyficznych operatorów matematycznych w API Metal skutkuje tzw. cichym przeniesieniem obliczeń na procesor główny (*ang. silent CPU fallback*). Może to powodować zatory w potoku przetwarzania. Ograniczenie to narzuca konieczność selekcji modeli i wyboru wyłącznie takich architektur, których wszystkie operacje są w pełni wspierane sprzętowo przez GPU/Neural Engine. Jak zostanie to szerzej omówione w dalszej części pracy, uwarunkowanie to bezpośrednio zdeterminowało odrzucenie najnowszych iteracji modeli YOLO w niniejszej pracy.

3.1.2 Środowisko programistyczne: Język Python oraz Miniforge

Jako główną warstwę aplikacyjną systemu MTMC wybrano język Python, funkcjonujący w wirtualnym środowisku Miniforge, będącym dystrybucją pakietów Conda dla architektury ARM64 [35]. Python stanowi obecnie standard w dziedzinie sztucznej inteligencji, zapewniając dostęp do najszerszego ekosystemu bibliotek analitycznych. Mimo relatywnie niskiej wydajności tego języka w zadaniach obliczeniowych, w kontekście nowoczesnych potoków głębokiego uczenia pełni on funkcję wysokopoziomowego interfejsu (*ang. wrapper*). Złożone obliczeniowo operacje matematyczne są delegowane do zoptymalizowanych, bibliotek zaimplementowanych w językach C/C++ i wykonywane bezpośrednio na dedykowanych akceleratorach sprzętowych.

Oficjalne wdrożenie pełnego wsparcia backendu MPS do najpopularniejszego frameworka uczenia maszynowego – *PyTorch* [3, 18], uzasadnia wybór tego rozwiązania. Dzięki temu ciężar operacji tensorowych może zostać przeniesiony bezpośrednio na układ graficzny procesora M5, za pomocą zaledwie jednej instrukcji w kodzie, definiując urządzenie sprzętowe jako `device = torch.device("mps")` [3]. Bezpośrednia integracja omija konieczność konwersji modeli do formatów pośrednich (np. CoreML) i pozwala na płynną współpracę z kluczowymi bibliotekami ekosystemu Python. Należą do nich m.in. NumPy – do wektoryzacji operacji w zunifikowanej pamięci UMA, SciPy – do rozwiązywania problemów asocjacyjnych na grafach dwudzielnych, oraz wbudowany moduł `multiprocessing`, wykorzystywany do izolowania procesów dekodowania strumieni H.264 z poszczególnych kamer.

Alternatywne środowiska programistyczne

Przez wzgląd na fakt, że system będzie działał na układach z rodziny Apple Silicon, zrezygnowano z wykorzystania języka C++ jako głównego środowiska programistycznego. W docelowych systemach wbudowanych dla monitoringu wizyjnego C++ gwarantuje najwyższą kontrolę i przewidywalność zarządzania stertą (*ang. heap*) oraz pamięcią podręczną, jednak na platformie macOS wymuszałoby to implementację procesów wnioskowania poprzez niskopoziomowe interfejsy lub adaptację lekkich silników brzegowych. Co więcej, implementacja najnowszych architektur opartych na transformato-
rach – które stanowią obecnie fundament nowoczesnych modułów reidentyfikacji obiektów [25] – w języku C++ i frameworku Metal znacznie wydłużyłaby czas programowania, nie przynosząc przy tym współmiernych zysków dla całkowitej wydajności systemu [18].

Z analogicznych przyczyn, podyktowanych przede wszystkim brakiem nowoczesnych, oficjalnie wspieranych przez społeczność dowiązań (*ang. bindings*) do interfejsu MPS, odrzucono również środowiska Java oraz MATLAB. Ponadto, jak uzasadniono we wcześniejszych podrozdziałach, odrzucono architekturę NVIDIA CUDA oraz TensorRT ze względu na ich niekompatybilność z układem SoC M5 [2].

3.2 Śledzenie wewnątrz pojedynczej kamery i detekcja obiektów

3.2.1 Ekosystem YOLO oraz modele RT-DETR

Do zadania lokalizacji obiektów w czasie niemal rzeczywistym wykorzystano jednoetapowe detektory (*ang. one-stage detectors*) zaimplementowane w bibliotece Ultralytics. Skupiono się przede wszystkim na architekturze YOLOv8 oraz modelach opartych na transformato-
rach wizyjnych z rodziny RT-DETR (*ang. Real-Time Detection Transformer*). Detektory jednoetapowe rozwiązują zadanie detekcji poprzez bezpośrednią regresję współrzędnych ramek i klasyfikacji w jednym przebiegu sieci (*ang. forward pass*), co zapewnia im znaczną przewagę wydajnościową nad klasycznymi podejściami opartymi na generowaniu regionów.

Z perspektywy wdrożenia systemu na układach Apple Silicon M5, kluczowym atutem biblioteki Ultralytics jest pełna integracja akceleracji sprzętowej poprzez bezpośredni dostęp do interfejsu Metal Performance Shaders (parametr `device='mps'`). Ważnym aspektem był jednak dobór odpowiedniej generacji algorytmu. Mimo dostępności w literaturze nowszych wersji takich jak YOLOv10 lub YOLOv11, ostatecznie zdecydowano się na wykorzystanie YOLOv8, w wariantach Nano oraz Medium, oraz modelu RT-DETR-L.

Decyzja ta wynika z dogłębnej analizy zachowania grafów obliczeniowych tych modeli na backendzie MPS [55]. Architektura YOLOv8 opiera się na podejściu bezkotwicowym (*ang. anchor-free*), rozdzielonej głowie detekcyjnej (*ang. split head*) oraz regresji dystrybucyjnej DFL (*ang. Distribution Focal Loss*). Jej operacje splotowe wykazują pełną zgodność z szablonami zoptymalizowanych warstw w Apple Metal. Z kolei nowsze architektury wykorzystują eksperymentalne operacje uwagi, których brak w bibliotece MPS skutkuje tzw. cichym przeniesieniem obliczeń na powolny procesor główny (CPU) [55]. Dzięki optymalnej kompatybilności, model YOLOv8-Nano uruchomiony na układach z serii Apple M wykazuje w testach średnie opóźnienie rzędu 210 ms (ok. 4,75 FPS), przy jednoczesnym bardzo stabilnym odchyleniu standardowym na poziomie 123 ms [55].

Z kolei dla scen charakteryzujących się bardzo wysokim zagęszczeniem pieszych i długotrwałymi okluzjami, klasyczne filtry bazujące na redukcji nadmiarowych ramek wykazują tendencję do błędnego odrzucania pokrywających się sylwetek [55]. W takich sytuacjach system hybrydowo integruje model RT-DETR-L. Formuluje on problem detekcji jako bezpośrednią predykcję zbioru obiektów (*ang. direct set prediction*), wykorzystując hybrydowy enkoder i algorytm optymalnego skojarzenia w grafie dwudzielnym. Eliminacja heurystycznego algorytmu NMS zwalnia znaczne zasoby procesora CPU [55]. Co więcej, dzięki optymalizacji operacji macierzowych, model ten osiąga wysoką precyzję rzędu 54,71% (mAP@0.5:0.95) przy stabilnym opóźnieniu 331 ms [55]. Wybór ten stanowi więc kompromis pomiędzy precyzją niezbędną do minimalizacji fragmentacji trajektorii, a opóźnieniem wymagany w systemach bezpieczeństwa.

3.2.2 Ograniczenia nowszych architektur YOLO w środowisku Apple MPS

Choć w środowiskach opartych na akceleratorach NVIDIA, modele YOLO v10 i v11 wykazują wyższą precyzję przy mniejszej liczbie teoretycznych operacji zmiennoprzecinkowych, badania empiryczne dowodzą, że wskaźniki te nie przekładają się na niskie opóźnienia na urządzeniach brzegowych (*ang. edge devices*), takich jak układy Apple Silicon [55]. Nowe generacje YOLO, w szczególności wersja v10, wprowadzają specyficzne moduły uwagi (*ang. Partial Self-Attention – PSA*). Ponieważ framework Metal Performance Shaders wciąż posiada ograniczony zakres wspieranych operatorów w porównaniu ze środowiskiem CUDA, kompilator grafu obliczeniowego nie jest w stanie optymalnie zinterpretować bloków PSA [55].

Z powodu braku odpowiedniego wzorca sprzętowego, środowisko PyTorch wykonuje tzw. ciche przeniesienie obliczeń (*ang. silent CPU fallback*) z wydajnych jednostek graficznych z powrotem na powolniejsze wątki procesora centralnego (CPU) [55, 2].

Z analogicznych względów wydajnościowych całkowicie zrezygnowano z detektorów dwuetapowych, takich jak architektura Faster R-CNN [53]. Opierają się one na procesie generowania propozycji regionów (RPN), którego iteracyjne przetwarzanie wyczerpałoby dopuszczalny budżet opóźnień (*ang. latency budget*) całego systemu. Z kolei klasyczne podejścia z dziedziny wizji komputerowej (np. deskryptory HOG + SVM czy kaskady Viola-Jones) zostały odrzucone z powodu ich niskiej odporności na zniekształcenia perspektywiczne oraz braku zdolności adaptacyjnych do luki domenowej (*ang. domain gap*).

3.2.3 Asocjacja danych i metody filtracji trajektorii: BoT-SORT i ByteTrack

Najważniejszym kryterium doboru algorytmu asocjacji w pojedynczej kamerze był kompromis pomiędzy odpornością na zjawisko okluzji a opóźnieniami w potoku przetwarzania. Choć klasyczny algorytm SORT [6] cechuje się wydajnością rzędu 260 Hz, przez podatność na utratę tożsamości obiektów w gęstych scenach nie mógł być zastosowany w systemie monitoringu [6]. Wprowadzenie do procesu asocjacji cech wizualnych w architekturze Deep SORT [70] pozwoliło wprowadzić na redukcję liczby błędów zmiany tożsamości (*ang. ID switches*) o 45%, jednak na współczesnych, zatłoczonych zbiorach danych model ten ustępuje precyzją nowszym rozwiązaniom [70].

Szczegółowej analizie porównawczej poddano architekturę ByteTrack [73] oraz jej udoskonalone rozwinięcie – BoT-SORT [1]. Średni czas wnioskowania ByteTrack wynosi 26,6 ms, jednak dla nietypowych warunków model ten traci precyzję lokalizacji przestrzennej [56]. Jak dowodzą najnowsze badania ewaluacyjne, dla obiektów szybko poruszających się lub często zasłanianych, błędy śledzenia ByteTracka - wyrażone w średnim przesunięciu względem celu - potrafią być o 260% wyższe niż w konkurencyjnych metodach, osiągając średnie odchylenie ADE na poziomie 114,3 piksela [56].

Z tego powodu ostatecznie zdecydowano się na wdrożenie algorytmu BoT-SORT [1]. Rozwiązuje on problem wysokiego błędu przestrzennego i wykazuje znacznie wyższą stabilność trajektorii przy wciąż akceptowalnym opóźnieniu. W testach na zbiorze MOT17, BoT-SORT osiągnął najlepsze wyniki skuteczności (MOTA na poziomie 80,6 oraz IDF1 na poziomie 79,5), wyprzedzając tym samym swojego poprzednika [1].

Docelowa integracja niskowariancyjnych predykcji detektora YOLOv8 z asocjatorem BoT-SORT, który z powodzeniem łączy w jednym kroku koszty geometryczne (IoU) i wizualne (Re-ID), gwarantuje rzetelną bazę do utrzymania ciągłości trajektorii. Stanowi to warunek konieczny dla niezawodnego działania globalnego modułu kojarzenia obiektów pomiędzy wieloma kamerami.

3.3 Reidentyfikacja obiektów (Re-ID)

3.3.1 Vision Transformers i TransReID

Ze względu na relatywnie niską rozdzielczość w systemach CCTV, która uniemożliwia identyfikację biometryczną (np. na podstawie cech twarzy), rozpoznawanie obiektów musi opierać się wyłącznie na elementach ubioru, geometrii oraz cechach szczególnych. Do implementacji modułu wybrano

ekosystem biblioteki `transformers` firmy Hugging Face, która posiada spójny interfejs API do eksperymentów badawczych i jest kompatybilna z akceleratorami Apple Silicon. Transformacja modeli odbywa się w kodzie w czasie rzeczywistym poprzez bezpośrednie lokowanie wag w architekturze Metal (metoda `.to("mps")`).

Jako architekturę docelową wybrano rozwiązanie inspirowane modelem TransReID [25], oparte o czyste transformatory wizyjne - ViT. Dzięki mechanizmowi samoatencji (*ang. self-attention*), architektury te wyodrębniają zależności przestrzenne dalekiego zasięgu bezpośrednio z sekwencji wycinków obrazu (*ang. patches*), zachowując przy tym pełną rozdzielczość niezbędną do odróżniania wizualnie podobnych obiektów [25]. Zrezygnowano z implementacji klasycznych architektur opartych na konwolucyjnych sieciach neuronowych (CNN), takich jak ResNet-50 ze schematem IDE czy popularny model PCB [59]. Zasadniczą wadą sieci CNN w zadaniach Re-ID jest bezpowrotna utrata drobnych, ale istotnych szczegółów sylwetki - logotypów na odzieży, krawędzi plecaków - wynikająca z postępującej redukcji rozdzielczości w operacjach łączenia (*ang. downsampling/pooling*).

3.3.2 Wybór metod adaptacji domenowej i optymalizacji przestrzeni cech

Wyzwaniem w monitoringu wielokamerowym pozostaje tzw. luka domenowa (*ang. domain gap*), objawiająca się drastycznymi różnicami w przestrzeni barw oraz perspektywie pomiędzy różnymi punktami obserwacyjnymi. Aby zneutralizować ten problem i uodpornić system na zmienne warunki środowiskowe, zdecydowano się na zastosowanie dwóch komplementarnych technologii.

Pierwszym zidentyfikowanym problemem były zniekształcenia geometryczne i perspektywiczne sylwetek. Do ich korekty wybrano mechanizm bocznych osadzeń informacyjnych (*ang. Side Information Embeddings - SIE*) [25]. Przez wzgląd na jego zdolność do bezpośredniego przekazywania identyfikatorów poszczególnych kamer do wektora reprezentacji modelu. W przeciwieństwie do klasycznych rozwiązań, SIE pozwala na jawną kompensację różnic wizualnych wynikających z unikalnego umiejscowienia kamery, ułatwiając sieci rozpoznanie tego samego obiektu pod różnymi kątami [25].

Drugim istotnym problemem projektowym był brak oznaczonych danych dla docelowego środowiska działania systemu, a ręczne etykietowanie nowych nagrań jest procesem wysoce nieefektywnym. W celu jego rozwiązania, proces optymalizacji oparto na architekturze nienadzorowanej adaptacji domenowej (UDA). Spośród dostępnych metod wytypowano schemat MMT (*ang. Mutual Mean-Teaching*) [20]. MMT wykazuje odporność na zaszumione dane – algorytm ten poprawia zdolności uogólniania modelu poprzez generowanie miękkich pseudoetykiet przy użyciu uśrednionych wag nauczyciela, skutecznie transferując wiedzę z domeny źródłowej do docelowej bez jakiegokolwiek nadzoru eksperta [20].

Do optymalizacji przestrzeni wyekstrahowanych cech na układzie Apple M5 wybrano funkcję straty trójkowej z trudną eksploracją (*ang. Batch-Hard Triplet Loss*) [27]. W przeciwieństwie do standardowych miar klasyfikacyjnych lub naiwnego doboru trójek, które prowadzą do szybkiej stagnacji uczenia, metoda ta wymusza na sieci minimalizowanie odległości euklidesowej D wyłącznie

dla najtrudniejszych przypadków pozytywnych i negatywnych w danej paczce:

$$\mathcal{L}_{\text{BH}} = \sum_{i=1}^P \sum_{a=1}^K \left[m + \overbrace{\max_{p=1\dots K} D(f_{\theta}(x_i^{(a)}), f_{\theta}(x_i^{(p)}))}^{\text{najtrudniejszy pozytyw}} - \overbrace{\min_{\substack{j=1\dots P \\ n=1\dots K \\ j \neq i}} D(f_{\theta}(x_i^{(a)}), f_{\theta}(x_j^{(n)}))}^{\text{najtrudniejszy negatyw}} \right]_+ \quad (22)$$

gdzie P oznacza liczbę unikalnych osób w paczce, K liczbę obrazów danej osoby, a m to założony margines separacji. Zastosowanie tej funkcji gwarantuje wyuczenie się wysoce dyskryminatywnych deskryptorów wizualnych, co bezpośrednio przekłada się na wyższą skuteczność kojarzenia trajektorii w globalnym module asocjacji [27].

Ostatecznie, wdrożenie tak zaawansowanych mechanizmów musiało zostać dostosowane do specyfiki sprzętowej procesora Apple M5. Ponieważ natywny backend MPS (*Metal Performance Shaders*) posiada przejściowe braki w implementacji niektórych niskopoziomowych operatorów macierzowych, architekturę uzupełniono o jawną deklarację zmiennej środowiskowej `os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"`. Decyzja ta była niezbędna do zabezpieczenia procesu ekstrakcji cech przed krytycznymi przerwaniem – mechanizm ten wymusza automatyczne i bezpieczne przeniesienie wykonania nieobsługiwanych fragmentów grafu obliczeniowego na procesor centralny (CPU), gwarantując nieprzerwaną pracę całego systemu.

3.3.3 Technologie odrzucone w zadaniu reidentyfikacji

Podczas projektowania modułu Re-ID świadomie odrzucono metody oparte na ręcznie projektowanych cechach wizualnych (*ang. hand-crafted features*), w tym zaawansowane deskryptory takie jak LOMO (oparte na transformatach Retinex i agregacji maksymalnych wystąpień w przesuwanych oknach) [38] oraz moduły symetryczne SDALF [4]. Choć charakteryzują się one znacznie niższym narzutem obliczeniowym niż architektura TransReID, ich skuteczność drastycznie spada w przypadku zaszumionych nagrań o niskiej rozdzielczości oraz przy znacznych zmianach perspektywy (*ang. viewpoint changes*) występujących pomiędzy różnymi kamerami. Zrezygnowano również z integracji klasycznych architektur syjamskich, trenowanych z wykorzystaniem funkcji straty kontrastowej w starszych frameworkach (takich jak Caffé czy Theano). Stanowią one obecnie przestarzałe rozwiązania, pozbawione możliwości płynnej adaptacji ich środowisk uruchomieniowych do architektury zunifikowanej pamięci układów Apple M5.

Ostatecznie nie zdecydowano się także na wdrożenie metod realizujących założenia *Privacy by Design*, czyli systemów bazujących na topologii szkieletowej (np. modele grafowe TranSG [51]) lub analizie dynamiki chodu (np. algorytm GaitSet [9]). Pomimo ich niewątpliwych zalet w kontekście anonimizacji wizerunku, odrzucenie to podyktowane było nieproporcjonalnie wysokim kosztem obliczeniowym niezbędnym do ekstrakcji punktów kluczowych sylwetki. Wymagałoby to uruchomienia ciężkich estymatorów pozy (np. OpenPose) całkowicie współbieżnie z istniejącymi już detektorami obiektów, co natychmiast wyczerpałoby dostępny budżet opóźnień całego systemu.

3.4 Geometria sceny, rzutowanie BEV i modelowanie tła poprzez bibliotekę OpenCV

3.4.1 OpenCV z optymalizacjami dla architektury ARM

Do realizacji podstawowych operacji na obrazach, dekodowania macierzy perspektywy oraz wstępnego przetwarzania surowych strumieni wideo, wybrano powszechnie stosowaną bibliotekę OpenCV (*ang. Open Source Computer Vision*) w postaci interfejsu programistycznego dla języka Python. Istotnym atutem tego rozwiązania w kontekście wdrożenia na układzie SoC Apple M5 jest natywna optymalizacja jej bazowego kodu C++ pod procesory z rodziny ARM [48].

Kluczowe algorytmy matematyczne OpenCV korzystają ze sprzętowych instrukcji wektorowych ARM NEON. Odbywa się to poprzez tzw. uniwersalne instrukcje wewnętrzne (*ang. universal intrinsics*), deklarowane m.in. w nagłówku `arm_neon.h`, które potrafią wektoryzować operacje w środowisku zredukowanych rozkazów RISC na 128-bitowych rejestrach [48]. Pozwala to na znaczną redukcję czasu wykonywania operacji zmiennoprzecinkowych (FP16) oraz całkowitoliczbowych (SIMD) podczas analizy każdej z milionów klatek rejestrowanego wideo. Dodatkowo, w środowisku macOS struktura OpenCV integruje się z niskopoziomowym pakietem Apple Accelerate Framework. Jest to zbiór zoptymalizowanych procedur dostarczających funkcje numeryczne, operacje algebry liniowej oraz biblioteki przetwarzania sygnałów. Przekłada się to na wysoką wydajność podstawowych przekształceń, takich jak konwersje przestrzeni barw, rzutowania perspektywiczne czy zmiany rozdzielczości, co skutecznie odciąża procesor główny, rezerwując pule cykli obliczeniowych dla złożonych asocjacji logicznych w procesie śledzenia.

W celu skutecznego rzutowania detekcji pomiędzy wieloma kamerami, zwłaszcza w scenach o dużym zagęszczeniu i silnych okluzjach, niezbędna jest transformacja geometrii wideo. W systemie wykorzystano w tym celu Probabilistyczne Mapy Zajętości (POM, *ang. Probabilistic Occupancy Map*), zaproponowane pierwotnie przez Fleureta i in. [19]. Algorytm ten szacuje prawdopodobieństwo zajętości danej komórki siatki na płaszczyźnie gruntu (reprezentowane zmienną $X_k \in \{0, 1\}$) przez pieszego. Estymacja ta opiera się bezpośrednio na maskach izolowanego tła we wszystkich dostępnych kamerach [19]. POM minimalizuje błędy rzutowania przy użyciu probabilistycznego przybliżenia wariacyjnego, redukując odległość Kullbacka-Leiblera pomiędzy estymowanym a rzeczywistym rozkładem prawdopodobieństwa obecności obiektów.

Zestawienie syntetycznego obrazu A^c , powstałego jako suma logiczna rzutowanych sylwetek ($A^c = \bigoplus_k X_k A_k^c$), z faktycznym obrazem masek tła B , pozwala na obliczenie kary (tzw. pseudoodległości) za wszelkie niezgodności z obrazem rzeczywistym [19]:

$$\Psi(B, A) = \frac{1}{\sigma} \frac{|B \otimes (1 - A) + (1 - B) \otimes A|}{|A|} \quad (23)$$

W powyższym równaniu operatory $\otimes$ oraz dodawania realizują operację różnicy symetrycznej (pomiędzy fałszywymi detekcjami a brakiem pokrycia), która następnie jest normalizowana przez powierzchnię $|A|$ i skalowana parametrem ufności σ .

Powyższy mechanizm, zwiększając odporność systemu na zjawisko wzajemnego przesłaniania się obiektów wzdłuż osi optycznej kamery, skutecznie koryguje fizyczne współrzędne predykcji. Jako dane wejściowe (zbiór B) do algorytmu POM podawane są maski pierwszoplanowe, uzyskane przy pomocy technik wyodrębniania tła (*ang. Background Subtraction – BGS*) natywnie zaimplementowanych w bibliotece OpenCV. W potoku przetwarzania wykorzystane zostały dwa podejścia. Pierwszym jest wariant estymacji nieparametrycznej KNN, bazujący na estymatorze balonowym, który ogranicza złożoność obliczeniową poprzez adaptację rozmiaru jądra do lokalnej gęstości danych [81]. Drugim jest ulepszony algorytm mieszanin gaussowskich MOG2 [80].

Zastosowanie algorytmu MOG2 skutecznie zwalnia system na sprzęcie brzegowym z wysokiego obciążenia pamięciowego (*ang. memory footprint*). Model ten przypisuje każdemu pikselowi obrazu mieszaninę rozkładów normalnych z automatycznym kryterium optymalizacji ich liczby. Ostateczne wagi w procesie mapowania MOG2 są aktualizowane z wykorzystaniem ujemnego współczynnika rozkładu a priori Dirichleta:

$$\hat{\pi}_m \leftarrow \hat{\pi}_m + \alpha(o_m^{(t)} - \hat{\pi}_m) - \alpha c_T \quad (24)$$

Równanie to gwarantuje szybkie usunięcie przestarzałych klastrów tła (zjawisko tzw. eliminacji komponentów), które przestają otrzymywać wzmocnienie informacyjne, pozwalając modelowi na płynną adaptację do zmian w scenie [80].

Całość operacji geometrycznych bazuje następnie na modelach kalibracyjnych i estymacji macierzy homografii 3×3 , które transformują punkty z obrazu perspektywicznego na płaszczyznę gruntu za pomocą klasycznej metody odwrotnego mapowania perspektywicznego (IPM, *ang. Inverse Perspective Mapping*) [41]:

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (25)$$

W środowisku OpenCV transformacja `cv::warpPerspective` jest bezpośrednio i natywnie wektoryzowana na sprzętowe jednostki SIMD (ARM NEON) w układzie M5, co pozwala na jej wykonanie w ułamku sekundy [48].

Z rozważań projektowych definitywnie wykluczono współczesne algorytmy transformacji do przestrzeni BEV oparte na głębokich sieciach neuronowych trenowanych w rygorze *end-to-end* – takie jak modele Lift, Splat, Shoot (LSS) [49] czy architektury bazujące na transformatorach atencyjnych (np. BEVFormer [37]). Mimo że w domenie pojazdów autonomicznych technologie te z powodzeniem wypierają modele analityczne (ignorując problem braku idealnej płaskości terenu), ich wysokie wymagania obliczeniowe oraz zapotrzebowanie na ogromne zbiory danych uczących czynią je niepraktycznymi w statycznym monitoringu wizyjnym (CCTV). Wymagałyby one ciągłego ewaluowania wielkich, trójwymiarowych grafów wyłącznie po to, by wygenerować

rzuty dla sztywno zamontowanych kamer. Z tak sformułowanym problemem algorytmy analityczne OpenCV radzą sobie natychmiastowo, korzystając ze stałych macierzy kalibracji. Odrzucono również starsze algorytmy wyodrębniania tła, takie jak Bayesowski GMG, gdyż wymagają one długiej fazy inicjalizacyjnej opóźniającej wejście systemu w tryb gotowości operacyjnej i wykazują niższą stabilność w środowiskach wbudowanych [48].

Rozdział 4

Implementacja systemu śledzenia i reidentyfikacji

4.1 Architektura potoku przetwarzania danych

Wdrożone rozwiązanie analizy nagrań z systemu wielokamerowego opiera się na kaskadowym potoku przetwarzania danych (*ang. pipeline*). Informacje wizualne są analizowane wieloetapowo: od lokalizacji przestrzennej, poprzez śledzenie wewnątrz pojedynczej kamery, aż po globalną asocjację tożsamości. Złożoność obliczeniowa równoległego przetwarzania kilku strumieni wideo w wysokiej rozdzielczości wymusiła zastosowanie architektury współbieżnej. System nie analizuje sekwencyjnie każdej kamery po kolei, lecz dystrybuje zadania na niezależne podprocesy.

4.1.1 Środowisko uruchomieniowe i optymalizacja dla układu Apple M5

Kluczowym uwarunkowaniem wdrożeniowym prezentowanego systemu była specyfika platformy sprzętowej – procesora z rodziny Apple Silicon M5. Układy te wykorzystują zunifikowaną architekturę pamięci (*ang. Unified Memory Architecture – UMA*). Oznacza to, że procesor główny oraz wielordzeniowy koprocesor graficzny współdzielą wspólną przestrzeń pamięci operacyjnej. Pozwala to na całkowite wyeliminowanie narzutu czasowego związanego z tradycyjnym kopiowaniem zdekodowanych klatek wideo z pamięci RAM do pamięci VRAM na magistrali PCIe.

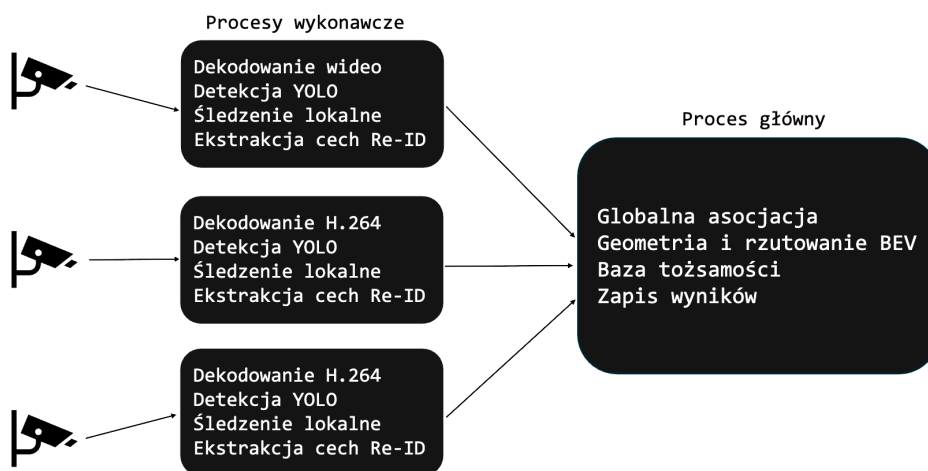
Aby wykorzystać potencjał obliczeniowy wbudowanych akceleratorów macierzowych, główny framework głębokiego uczenia – PyTorch został zintegrowany z interfejsem MPS. Alokacja modeli neuronowych w pamięci koprocesora odbywa się poprzez jawną deklarację urządzenia sprzętowego: `device = torch.device("mps")`. Ponieważ interfejs API Metal znajduje się w fazie intensywnego rozwoju i nie posiada natywnych implementacji dla wszystkich zaawansowanych operatorów wykorzystywanych w architekturach Transformer, wprowadzono zabezpieczenie na poziomie systemu operacyjnego. Poprzez deklarację zmiennej środowiskowej `os.environ["PYTORCH_ENABLE_MPS_FALLBACK"] = "1"` wymuszono automatyczne przekierowa-

nie nieobsługiwanych instrukcji na procesor główny, co gwarantuje bezawaryjną pracę potoku przetwarzania.

4.1.2 Wielowątkowość i asynchroniczna wymiana komunikatów

Zarządzanie strumieniami wideo z poszczególnych kamer zrealizowano przy użyciu wbudowanej biblioteki `multiprocessing`. Z uwagi na architekturę systemu macOS, do uruchamiania nowych procesów wykorzystano metodę `spawn`, co zapobiega konfliktom współdzielenia zasobów sprzętowych (GPU) podczas duplikowania grafów obliczeniowych.

Każda kamera obsługiwana jest przez niezależny proces roboczy (`camera_worker`). Proces ten samodzielnie dekoduje sygnał H.264, wykonuje detekcję oraz ekstrahuje cechy wizualne. Cechą wyróżniającą zaproponowanej architektury jest model komunikacji międzyprocesowej (IPC). Do głównego procesu asocjacji (Global Trackera) nie są przesyłane surowe klatki wideo, lecz wyłącznie ustrukturyzowane metadane w postaci słowników w języku Python. Pakiety te zawierają znormalizowane wektory cech Re-ID, wyliczone współrzędne na płaszczyźnie oraz identyfikator kamery. Takie podejście drastycznie redukuje obciążenie magistrali pamięciowej, umożliwiając łatwą skalowalność systemu poprzez dodawanie kolejnych sensorów wizyjnych.



Rysunek 2. Wizualizacja architektury potoku przetwarzania danych. Źródło: opracowanie własne.

4.2 Moduł detekcji i śledzenia w pojedynczej kamerze (SCT)

Realizacja śledzenia wieloobektowego opiera się w systemie na dominującym paradygmacie śledzenia przez detekcję (*ang. tracking-by-detection*). Zgodnie z tym podejściem problem rozdzielony jest na niezależną lokalizację przestrzenną obiektów oraz późniejsze łączenie ich w ciągłe trajektorie czasowe.

4.2.1 Inicjalizacja i wnioskowanie modelu YOLO

Jako fundament modułu lokalizacji przestrzennej wybrano model z rodziny YOLOv8 w wariantcie Medium (YOLOv8m). Decyzja ta podyktowana była dogłębną analizą wydajnościową w środowisku Apple MPS. W odróżnieniu od nowszych architektur zawierających moduły uwagi PSA, podstawowe bloki konwolucyjne modelu YOLOv8 są w pełni sprzętowo wspierane przez koprocesory M5, zapobiegając drastycznym skokom opóźnień (*latency spikes*). Zastosowany wariant Medium oferuje korzystny kompromis pomiędzy czasem wnioskowania a precyzją wykrywania częściowo zasłoniętych sylwetek w zagęszczonych scenach miejskich.

Dla optymalizacji obciążenia logiki decyzyjnej proces detekcji poddano rygorystycznej filtracji wejściowej. Próg ufności (*confidence score*) sieci splotowej ustalono na poziomie 25%, odrzucając predykcje o niskim stopniu pewności. Następnie nałożono filtr semantyczny, redukując wyjściową przestrzeń wyników ze wszystkich 80 klas zbioru COCO wyłącznie do klasy 0, odpowiadającej detekcji pieszego. Wynikiem działania sieci jest wektor współrzędnych ramki otaczającej $b = [x_1, y_1, x_2, y_2]^T$, na podstawie którego wyznaczany jest centralny punkt dolnej krawędzi reprezentujący fizyczny styk obiektu z podłożem.

4.2.2 Śledzenie lokalne z wykorzystaniem algorytmu BoT-SORT

Aby odfiltrować błędne detekcje oraz zniwelować efekty krótkotrwałych okluzji (np. przejście pieszego za znakiem drogowym), wewnątrz każdego procesu kamery zaimplementowano lokalny algorytm asocjacji BoT-SORT. Model ten wprowadza istotne modyfikacje względem klasycznych rozwiązań, bezpośrednio modelując szerokość i wysokość ramki otaczającej w strukturze filtru Kalmana.

Filtr Kalmana w każdej klatce estymuje przyszłe położenie obiektu na podstawie jego dotychczasowej prędkości wektorowej. BoT-SORT wykorzystuje algorytm kompensacji ruchu (CMC) do niwelowania ewentualnych drgań obrazu, a następnie wyznacza macierz kosztów asocjacji. Działając jako bufor wygładzający, śledzenie lokalne grupuje dyskretne detekcje pieszych w spójne fragmenty trajektorii (*ang. tracklets*). Pozwala to na wystanie do globalnego modułu wiarygodniejszej informacji o tożsamości, odciążając tym samym wyższe warstwy systemu.

4.3 Ekstrakcja cech wizualnych i reidentyfikacja (Re-ID)

Ograniczone pole widzenia pojedynczego sensora wymusza przekazanie odpowiedzialności za ciągłość trajektorii obiektów poruszających się między kamerami do dedykowanego modułu reidentyfikacji (Re-ID). Zadaniem tego etapu jest wyznaczenie unikalnej sygnatury wizualnej, odpornej na zmiany perspektywy i warunków oświetleniowych.

4.3.1 Implementacja architektury Vision Transformer (ViT)

Zrezygnowano z zastosowania klasycznych sieci splotowych (takich jak powszechnie stosowane warianty ResNet). Operacje zmniejszania rozdzielczości (*pooling*) w głębokich warstwach sieci CNN

Tabela 2. Parametry detektora i trackera lokalnego. Źródło: opracowanie własne.

Parametr	Wartość	Opis
Akceleracja sprzętowa	Apple MPS	Wymuszenie wnioskowania na GPU procesora M5
Model detekcji	YOLOv8m	Wersja o optymalnym stosunku wydajności do precyzji
Classes	0	Ograniczenie detekcji do klasy pieszych
Próg ufności - conf	0.25	Minimalne prawdopodobieństwo uznania ramki otaczającej za reprezentującą rzeczywisty obiekt.
Próg pokrycia - iou	0.50	Próg geometrycznego pokrycia ramek dla algorytmu asocjacji oraz redukcji duplikatów (NMS).
Tracker lokalny	BoT-SORT	Zintegrowany algorytm śledzenia generujący krótkie trajektorie wewnątrz pojedynczej kamery.
Wiek ścieżki - max_age	30	Maksymalna liczba klatek, przez które tracker podtrzymuje tożsamość ukrytego/zgubionego obiektu.

prowadzą do bezpowrotnej utraty wysokoczęstotliwościowych detali, takich jak wzory odzieży czy krawędzie plecaków, co jest wysoce niepożądane przy identyfikacji osób o podobnym ogólnym zarysie sylwetki.

W odpowiedzi na te ograniczenia zaimplementowano architekturę opartą na transformatorach wizyjnych (Vision Transformer - ViT), czerpiąc z rozwiązań zaproponowanych w modelu TransReID. Mechanizm samoatencji (*self-attention*) dzieli wejściowy wycinek ramki pieszego na równe fragmenty (*patches*), modelując zależności przestrzenne o dalekim zasięgu pomiędzy nimi na przestrzeni wszystkich warstw, co znacznie zwiększa odporność na częściowe zasłonięcia sylwetki.

4.3.2 Generowanie i normalizacja wektorów cech (Embeddings)

Wykorzystując biblioteki ekosystemu Hugging Face, wykadrowana sylwetka pieszego jest formatowana do wymagań wejściowych modelu (zmiana rozmiaru do 224×224 pikseli, konwersja przestrzeni barw z domyślnej dla OpenCV BGR na RGB). Proces wnioskowania (*forward pass*) realizowany jest w wysoce zoptymalizowanym kontekście `torch.no_grad()`, co zapobiega alokacji pamięci na obliczanie gradientów.

Rezultatem operacji jest tensor zagregowanej reprezentacji cech całej sylwetki (tzw. *embedding*) o wymiarach 1×768 . Uzyskany wektor v poddawany jest na wyjściu normalizacji L2 według wzoru:

$$v_{\text{norm}} = \frac{v}{\|v\|_2}$$

Operacja ta nakłada wektory cech na jednostkową sferę wielowymiarową, co jest warunkiem koniecznym do poprawnego zastosowania odległości cosinusowej w procesie globalnej oceny podobieństwa.

4.4 Globalna asocjacja danych (Multi-Camera Association)

Wielowątkowy potok danych zbiega się w obiekcie klasy `GlobalTracker`, pełniącej rolę nadrzędnego zarządcy tożsamości. Głównym wyzwaniem tego modułu jest matematyczne udowodnienie, że sylwetki wyekstrahowane z różnych kamer w różnym czasie należą do tego samego pieszego.

4.4.1 Konstrukcja macierzy kosztów i łączenie błędów

Algorytm w każdej klatce buduje globalną macierz odległości, oceniając stopień dopasowania wszystkich nowo nadesłanych detekcji względem obiektów istniejących w bazie. Problem reidentyfikacji rozszerzono o weryfikację fizyczną, tworząc hybrydową macierz kosztów, na którą składają się:

1. **Koszt wizualny:** Wyliczany przy pomocy zoptymalizowanej funkcji z biblioteki `SciPy`, polegający na wyznaczeniu odległości cosinusowej pomiędzy znormalizowanymi wektorami `Re-ID`. Odległość bliska zeru oznacza skrajnie wysoką zbieżność tekstur i proporcji ubioru.
2. **Koszt przestrzenny:** Metryka euklidesowa wyznaczana na podstawie pozycji obiektów przeniesionych na globalną płaszczyznę mapy rzutu pionowego (`Bird's-Eye View`). Weryfikacja dokładnego procesu transformacji współrzędnych z ujęcia kamery na mapę `BEV` zostanie szczegółowo omówiona w rozdziale 5.

W celu redukcji obciążenia obliczeniowego wprowadzono zaporowe progi odcięcia (np. `max_visual_cost`). Jeśli badana para przekracza ustalony dystans euklidesowy lub cosinusowy, przypisywany jest jej ryczałowo skrajnie wysoki koszt (tzw. koszt zaporowy), co natychmiast eliminuje ją z grafu poszukiwań jako parę fizycznie niemożliwą.

4.4.2 Dopasowanie z użyciem algorytmu węgierskiego

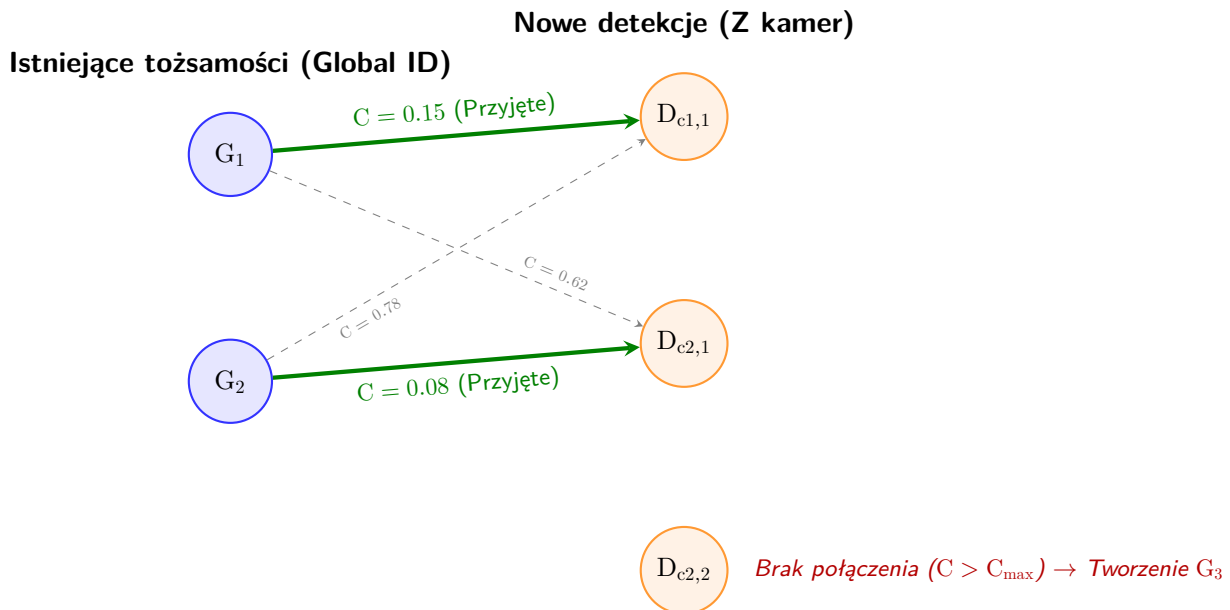
Wygenerowana hybrydowa macierz kosztów stanowi wejście dla dyskretnej optymalizacji algorytmu węgierskiego (metody Kuhna-Munkresa). Wykorzystano w tym celu implementację `linear_sum_assignment` pakietu `scipy.optimize`. Algorytm ten rozwiązuje problem asocjacji w grafie dwudzielnym w czasie wielomianowym, wyznaczając takie relacje "jeden do jednego", które minimalizują globalną sumę przypisań dla całego skrzyżowania. Stanowi to kluczowy element radzenia sobie z problemem martwych stref między polami widzenia, gwarantując najmniejszy błąd asocjacji, nawet gdy w obrębie ujęć kilku kamer pojawiają się wizualnie podobni przechodnie.

4.4.3 Zarządzanie cyklem życia globalnych identyfikatorów

Ostatnim etapem procesu jest zatwierdzenie wyników asocjacji oraz aktualizacja grafu tożsamości. Pomyślnie dopasowane detekcje natychmiast odświeżają wektory cech oraz globalne pozycje fizyczne w obiekcie `GlobalTracker`.

Zarządzanie trajektoriami jest silnie deterministyczne. Nowe wykrycia, dla których algorytm węgierski nie zdołał dopasować istniejącej historii, rejestrowane są jako nowe globalne identyfikatory

(*Global ID*). Równolegle zastosowano mechanizm obsługi tzw. wygasających ścieżek. W sytuacji, gdy przechodzień trwale opuści pole widzenia wszystkich kamer monitoringu, jego identyfikator nie jest od razu porzucany. System inkrementuje licznik brakujących klatek (*missed_frames*), przechowując tożsamość w stanie uśpienia. Dopiero po przekroczeniu ustalonego bufora czasowego, struktury przechowujące jego wektory cech są bezpowrotnie zwalniane z pamięci operacyjnej, zapobiegając przepełnieniu zunifikowanej pamięci RAM układu.



Rysunek 3. Ilustracja procesu globalnej asocjacji danych za pomocą grafu dwudzielnego. Algorytm węgierski szuka takich połączeń (pogrubione zielone linie), które minimalizują sumę kosztów C opartych na odległości Re-ID oraz geometrii BEV. Detekcja $D_{c2,2}$ nie spełnia progów podobieństwa i inicjuje nowy identyfikator G_3 .

Rozdział 5

Integracja widoku wielokamerowego i rzutowanie na mapę

Zastosowanie pojedynczych kamer w systemach wizyjnych dostarcza precyzyjnych informacji o lokalizacji obiektów wyłącznie w dwuwymiarowej przestrzeni obrazu (wyrażonej w pikselach). Rozwiązanie problemu śledzenia obiektów w rozproszonej sieci kamer (MTMC) wymaga sprowadzenia tych niezależnych obserwacji do wspólnego układu odniesienia. W niniejszym rozdziale przedstawiono implementację transformacji perspektywicznej do układu rzutu pionowego, powszechnie określanego jako widok z lotu ptaka (*ang. Bird's-Eye View* – BEV). Opisano również metody weryfikacji przestrzennej oparte na probabilistycznej mapie zajętości (POM), które korygują błędy rzutowania wynikające z okluzji w gęstym ruchu miejskim

5.1 Wyznaczanie i aplikacja macierzy homografii

Fundamentem integracji geometrycznej w zaimplementowanym systemie jest technika odwrotnego mapowania perspektywicznego (*ang. Inverse Perspective Mapping* – IPM). Przyjmując założenie, że obserwowany obszar stanowi płaską płaszczyznę terenu ($z = 0$), pełny model projekcji kamery redukuje się do płaskiego przekształcenia rzutowego, opisywanego przez macierz homografii H o wymiarach 3×3 .

5.1.1 Model geometryczny i odwrotne mapowanie perspektywiczne (IPM)

Zależność pomiędzy punktem na płaszczyźnie obrazu z kamery a jego rzeczywistym położeniem na globalnej mapie BEV opisana jest równaniem:

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

gdzie (x, y) to współrzędne pikselowe w obrazie źródłowym, (x', y') reprezentują skorygowane współrzędne w układzie mapy, a λ stanowi współczynnik skali układu współrzędnych jednorodnych.

W systemie proces ten jest aplikowany do punktu reprezentującego środek dolnej krawędzi ramki otaczającej, stanowiącego estymowany punkt styku pieszego z podłożem. Do realizacji przekształcenia w czasie rzeczywistym wykorzystano funkcję `cv2.perspectiveTransform` z biblioteki OpenCV. Dzięki natywnej optymalizacji biblioteki i integracji z pakietem Apple Accelerate na architekturze układu M5, operacje te ulegają wektoryzacji i są wykonywane w ułamkach milisekund, nie obciążając głównego wątku analitycznego.

> ****Wskazówka graficzna 1 (Rzutowanie IPM):**** W tym miejscu rekomendowane jest wstawienie grafiki składającej się z dwóch obok siebie położonych obrazów. Po lewej widok z pojedynczej kamery z naniesionym wielokątem (np. obrysowanym obszarem skrzyżowania). Po prawej stronie idealnie płaska mapa z góry (BEV), na którą ten sam wielokąt jest przekształcony w prostokąt. Pomiędzy obrazami strzałka z podpisem "Homografia H".

5.2 Ograniczenia rzutowania i identyfikacja problemu okluzji

Wykorzystanie statycznej macierzy homografii dostarcza wysoce precyzyjnych wyników pod warunkiem, że estymowany punkt referencyjny faktycznie leży na płaszczyźnie gruntu. Zjawiska występujące w zatłoczonym środowisku miejskim drastycznie naruszają to założenie, prowadząc do znaczących odchyśleń w wyznaczaniu globalnej pozycji.

5.2.1 Zjawisko błędu paralaksy w gęstym ruchu

Gdy przechodnie poruszają się w zwartych grupach, dochodzi do wzajemnego przesłaniania się obiektów (okluzji). W takiej sytuacji moduł detekcji YOLOv8 wyznacza dolną krawędź ramki otaczającej dla obiektu zasłoniętego na wysokości ciała osoby znajdującej się bliżej obiektywu, a nie na rzeczywistym podłożu. Rzutowanie takiego punktu za pomocą macierzy IPM prowadzi do tzw. błędu paralaksy – sztucznego odsunięcia obiektu na mapie w kierunku osi optycznej kamery. Konsekwencją tego błędu jest drastyczne zafałszowanie odległości euklidesowych pomiędzy pieszymi w module GlobalTracker, co prowadzi do błędnego działania algorytmu asocjacji i niezamierzonego łączenia różnych tożsamości. Wymusiło to konieczność implementacji niezależnej weryfikacji zajętości przestrzeni opartej na fizycznym ruchu.

5.3 Modelowanie tła i ekstrakcja fizycznego ruchu

Aby zweryfikować, czy obszar zadeklarowany przez moduł głębokiego uczenia jest faktycznie zajęty przez pieszego, zaimplementowano proces wyodrębniania ruchomego pierwszego planu. Wykorzystano do tego klasyczne techniki analizy tła (*ang. Background Subtraction – BGS*).

5.3.1 Inicjalizacja algorytmu mieszanin gaussowskich (MOG2)

Do detekcji zmian w pikselach wykorzystano algorytm `cv2.createBackgroundSubtractorMOG2`. W odróżnieniu od podejść opartych na gęstych sieciach głębokich, MOG2 charakteryzuje się bardzo niskim narzutem na pamięć operacyjną, modelując każdy piksel jako adaptacyjną mieszaninę rozkładów normalnych. Mechanizm automatycznej eliminacji komponentów gaussowskich (oparty na ujemnym współczynniku rozkładu a priori Dirichleta) skutecznie uodparnia maskę binarnego ruchu na wolno zmieniające się warunki oświetleniowe oraz dynamiczne tło, takie jak drgająca na wietrze roślinność

5.3.2 Przetwarzanie morfologiczne masek binarnego ruchu

Surowy obraz wygenerowany przez moduł MOG2 jest podatny na występowanie szumu pomiarowego w postaci izolowanych, pojedynczych pikseli. Z tego względu zaimplementowano krok przetwarzania morfologicznego. Maskę ruchu poddawana jest operacji otwarcia morfologicznego (`cv2.morphologyEx` z flagą `cv2.MORPH_OPEN`), co matematycznie odpowiada operacji erozji, po której następuje dyatacja. Zastosowanie odpowiednio dobranego elementu strukturalnego pozwala na usunięcie szumu oraz wygładzenie krawędzi kinetycznych, generując czysty, rzeczywisty obraz ruchu fizycznego na scenie.

Tabela 3. Parametry konfiguracyjne modułu analizy tła MOG2 i operacji morfologicznych. Źródło: opracowanie własne.

Parametr	Wartość	Opis wpływu na system
history	500	Rozmiar okna czasowego (liczba klatek) wpływającego na modelowanie rozkładów tła.
varThreshold	16.0	Próg wariancji decydujący o zaklasyfikowaniu piksela do przestrzeni ruchu.
detectShadows	True	Umożliwia identyfikację i ignorowanie cieni rzuconych przez pieszych na jezdnię.
Element strukturalny	3×3	Macierz (kernel) stosowana do operacji morfologicznego otwarcia maski ruchu.

5.4 Implementacja fuzji probabilistycznej mapy zajętości (POM)

Wiedza pozyskana z detektora YOLO (oparta na głębokich cechach semantycznych) oraz maski ruchu z MOG2 (oparta na zmianach pikseli) ulega fuzji w module Probabilistycznej Mapy Zajętości (POM). Implementacja tego etapu w kodzie realizuje matematyczne założenia minimalizacji błędu pomiędzy rzutem syntetycznym a obrazem rzeczywistym.

5.4.1 Generowanie rzutów syntetycznych (Obraz A)

Zgodnie z koncepcją POM, w pierwszej kolejności system tworzy syntetyczny obraz A. W pamięci operacyjnej inicjowana jest pusta (czarna) macierz numpy .zeros odpowiadająca rozdzielczości klatki kamery. Następnie skrypt iteruje po wszystkich ramach otaczających wygenerowanych przez model YOLO. W miejscu wystąpienia predykcji, na czarnej planszy rysowane są białe, wypełnione prostokąty (cv2.rectangle). Stanowi to matematyczną hipotezę modułu AI o geometrycznym rozkładzie zajętości w danej klatce.

5.4.2 Mechanizm weryfikacji karania za niezgodność (Operacja XOR)

Wygenerowany syntetyczny obraz rzutów (A) zestawiany jest ze skorygowaną maską fizycznego ruchu pochodzącą z modułu MOG2 (B). Zgodnie z literaturą, miara pseudoodległości wykorzystywana do weryfikacji zajętości komórek definiowana jest równaniem:

$$\Psi(B, A) = \frac{1}{\sigma} \frac{|B \otimes (1 - A) + (1 - B) \otimes A|}{|A|}$$

Licznik powyższego równania stanowi reprezentację matematycznej różnicy symetrycznej (alternatywy wykluczającej). Aby zapewnić wydajność potoku, operację tę zaimplementowano z wykorzystaniem funkcji cv2.bitwise_xor.

Po nałożeniu obu masek piksele, które pokrywają się w obu obrazach (YOLO deklaruje człowieka i MOG2 rejestruje w tym miejscu ruch), są uznawane za zweryfikowane geometrycznie. Z kolei obszary konfliktu (np. ramka YOLO "wisząca" w powietrzu na skutek błędu paralaksy, gdzie MOG2 zgłasza statyczne tło) są identyfikowane i nakładana jest na nie kara geometryczna. Powoduje to odrzucenie błędnie wyestymowanego styku obiektu z podłożem, korygując ostateczne mapowanie do przestrzeni BEV.

> ****Wskazówka graficzna 2 (Ilustracja procesu POM - Różnica Symetryczna):**** Jest to idealne miejsce na dodanie zestawienia 3 zminiaturyzowanych kadrów wideo. > 1. Obraz "A": Czarne tło i na nim idealne białe kwadraty w miejscach detekcji YOLO. > 2. Obraz "B": Surowy, zaszumiony, rzeczywisty biało-czarny obraz ruchu wygenerowany przez MOG2. > 3. Wynik (Operacja XOR): Obraz prezentujący tylko błędy (piksele konfliktu), które pozwalają algorytmowi skorygować błąd rzutu.

5.5 Fuzja w module asocjacji i globalna mapa BEV

Informacje geometryczne, pomyślnie przetransformowane oraz skorygowane przez mechanizm POM, przekazywane są do modułu głównego (GlobalTracker) jako argumenty do weryfikacji tożsamości.

5.5.1 Kalkulacja kosztu przestrzennego

Skorygowane współrzędne z lotu ptaka służą bezpośrednio do obliczenia dystansu przestrzennego na mapie MTMC. Zastosowano standardową metrykę euklidesową:

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Wartość ta stanowi komponent wagowy w ogólnej macierzy kosztów asocjacji (omawianej w poprzednim rozdziale), będąc bezwzględnym weryfikatorem poprawności łączenia tożsamości. Ustalenie fizycznego dystansu chroni system przed "teleportowaniem" tożsamości pieszego do innej części skrzyżowania wyłącznie na podstawie zbieżności ubioru.

5.5.2 Renderowanie ostatecznej, zunifikowanej przestrzeni MTMC

Zwieńczeniem całego potoku przetwarzania jest wygenerowanie metrycznej, cyfrowej reprezentacji obserwowanego obszaru w czasie rzeczywistym. System posiada dynamicznie alokowaną siatkę zajętości, na której nanosi weryfikowane globalne tożsamości pieszych w postaci punktów. Każda naniesiona trajektoria wzbogacona jest o metadane (Globalne ID oraz macierzysty węzeł kamery). Wynikowy rzut z lotu ptaka nie tylko ułatwia wizualną inspekcję poprawności działania całego systemu kalibracyjnego i reidentyfikacyjnego, ale przede wszystkim dostarcza znormalizowanych danych gotowych do analiz i ewaluacji za pomocą dedykowanych metryk statystycznych, takich jak np. ewaluator TrackEval.

Rozdział 6

Badanie i analiza efektywności systemu

Rozdział 7

Podsumowanie i wnioski

Bibliografia

- [1] Aharon, N., Orfaig, R. i Bobrovsky, B.-Z., „BoT-SORT: Robust Associations Multi-Pedestrian Tracking”, *arXiv preprint arXiv:2206.14651*, 2022.
- [2] Aliyev, M., „Benchmarking YOLOv8-Tiny for Real-Time Object Detection on macOS: A Comparison of Metal and CUDA Performance”, sierpień 2025.
- [3] Apple Inc., *Accelerated PyTorch training on Mac*, 2024. URL: <https://developer.apple.com/metal/pytorch/>
- [4] Bazzani, L., Cristani, M. i Murino, V., „Symmetry-driven accumulation of local features for human characterization and re-identification”, *Computer Vision and Image Understanding*, t. 117, nr 2, s. 130–144, 2013.
- [5] Bernardin, K. i Stiefelhagen, R., „Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics”, *EURASIP Journal on Image and Video Processing*, t. 2008, s. 1–10, 2008. DOI: 10.1155/2008/246309
- [6] Bewley, A., Ge, Z., Ott, L., Ramos, F. i Upcroft, B., „Simple online and realtime tracking (SORT)”, w: *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2016, s. 3464–3468.
- [7] Bouwmans, T., „Traditional and recent approaches in background subtraction for video surveillance: A systematic review”, *Computer Science Review*, t. 11, s. 31–66, 2014.
- [8] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A. i Zagoruyko, S., „End-to-End Object Detection with Transformers”, w: *Computer Vision - ECCV 2020*, Springer, 2020, s. 213–229. DOI: 10.1007/978-3-030-58452-8_13
- [9] Chao, H., He, Y., Zhang, J. i Feng, J., „GaitSet: Regarding Gait as a Set for Cross-View Gait Recognition”, w: *Proceedings of the AAAI Conference on Artificial Intelligence*, t. 33, 2019, s. 8126–8133.
- [10] Chen, C., Yang, Y., Zhou, J., Li, X. i Bao, F. S., „Cross-Domain Review Helpfulness Prediction Based on Convolutional Neural Networks with Auxiliary Domain Discriminators”, w: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, Walker, M., Ji, H. i Stent, A., red., New Orleans, Louisiana: Association for Computational Linguistics, 2018, s. 602–607. DOI: 10.18653/v1/N18-2095 URL: <https://aclanthology.org/N18-2095/>

- [11] Cheng, C.-T., Aslanov, M. i Kanjo, E., *Tiny Collaborative Inference for Occlusion-Robust Object Detection*, 2026. arXiv: 2606.02894 [cs.CV]. URL: <https://arxiv.org/abs/2606.02894>
- [12] Cherdchusakulchai, R. i in., „Online Multi-camera People Tracking with Spatial-temporal Mechanism and Anchor-feature Hierarchical Clustering”, w: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2024, s. 7198–7207. DOI: 10.1109/CVPRW63382.2024.00715
- [13] Cucchiara, R., Grana, C., Piccardi, M. i Prati, A., „Detecting moving objects, ghosts, and shadows in video streams”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 25, nr 10, s. 1337–1342, 2003.
- [14] Dalal, N. i Triggs, B., „Histograms of Oriented Gradients for Human Detection”, w: *Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, t. 1, 2005, s. 886–893. DOI: 10.1109/CVPR.2005.177
- [15] Dendorfer, P. i in., „MOT20: A benchmark for multi object tracking in crowded scenes”, *arXiv preprint arXiv:2003.09003*, 2020.
- [16] Elgammal, A., Harwood, D. i Davis, L. S., „Non-parametric model for background subtraction”, w: *Proceedings of the European Conference on Computer Vision*, 2000, s. 751–767.
- [17] Farenzena, M., Bazzani, L., Perina, A., Murino, V. i Cristani, M., „Person Re-Identification by Symmetry-Driven Accumulation of Local Features”, w: *Proceedings of the 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2010, s. 2360–2367. DOI: 10.1109/CVPR.2010.5539945
- [18] Feng, D., Xu, Z., Wang, R. i Lin, F. X., *Profiling Apple Silicon Performance for ML Training*, 2025. arXiv: 2501.14925 [cs.PF]. URL: <https://arxiv.org/abs/2501.14925>
- [19] Fleuret, F., Berclaz, J., Lengagne, R. i Fua, P., „Multicamera people tracking with a probabilistic occupancy map”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 30, nr 2, s. 267–282, 2008.
- [20] Ge, Y., Zhu, F., Chen, D., Zhao, R. i Li, H., „Mutual Mean-Teacher for Unsupervised Domain Adaptive Object Re-identification”, w: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [21] Girshick, R., „Fast R-CNN”, w: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, s. 1440–1448. URL: http://www.cv-foundation.org/openaccess/content_iccv_2015/papers/Girshick_Fast_R-CNN_ICCV_2015_paper.pdf
- [22] Girshick, R., Donahue, J., Darrell, T. i Malik, J., „Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation”, w: *Proceedings of the 2014 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014, s. 580–587. DOI: 10.1109/CVPR.2014.81
- [23] Gonzalez, R. C. i Woods, R. E., *Digital Image Processing (4th Edition)*. Pearson, 2018, ISBN: 978-1-292-22304-9.

-
- [24] Hartley, R. i Zisserman, A., *Multiple View Geometry in Computer Vision (2nd Edition)*. Cambridge University Press, 2004, ISBN: 978-0-511-18618-9.
- [25] He, S., Luo, H., Wang, P., Wang, F., Li, H. i Jiang, R.-R., „TransReID: Transformer-based Object Re-Identification”, w: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, s. 15 013–15 022.
- [26] He, X., Li, X. i Guo, X., *Differential Alignment for Domain Adaptive Object Detection*, 2024. arXiv: 2412.12830 [cs.CV]. URL: <https://arxiv.org/abs/2412.12830>
- [27] Hermans, A., Beyer, L. i Leibe, B., „In Defense of the Triplet Loss for Person Re-Identification”, *arXiv preprint arXiv:1703.07737*, 2017.
- [28] Jeong, J. i Kim, A., „Adaptive Inverse Perspective Mapping for lane map generation with SLAM”, w: *2016 13th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI)*, 2016, s. 38–41. DOI: 10.1109/URAI.2016.7734016
- [29] Jocher, G., Chaurasia, A. i Qiu, J., *Ultralytics YOLOv8*, GitHub repository, 2023. URL: <https://github.com/ultralytics/ultralytics>
- [30] Kalman, R. E., „A New Approach to Linear Filtering and Prediction Problems”, *Journal of Basic Engineering*, t. 82, nr 1, s. 35–45, 1960. DOI: 10.1115/1.3662552
- [31] Karbowski, L. i Bobulski, J., „Background Substraction in difficult weather conditions”, *PeerJ Computer Science*, 2022.
- [32] Krizhevsky, A., Sutskever, I. i Hinton, G. E., „ImageNet Classification with Deep Convolutional Neural Networks”, w: *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012, s. 1097–1105. URL: <http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf>
- [33] Kruegle, H., *CCTV Surveillance: Video Practices and Technology*. Butterworth-Heinemann, 2007, ISBN: 978-0-7506-7768-4.
- [34] Kuhn, H. W., „The Hungarian Method for the Assignment Problem”, *Naval Research Logistics Quarterly*, t. 2, nr 1–2, s. 83–97, 1955. DOI: 10.1002/nav.3800020109
- [35] Kumaravel, A., *How to Use Your MacBook Pro GPU for PyTorch (Apple Silicon)*, 2026. URL: <https://arul.co/how-to-use-your-macbook-pro-gpu-for-pytorch-apple-silicon-892b5130224b>
- [36] Li, H. i in., *Delving into the Devils of Bird's-eye-view Perception: A Review, Evaluation and Recipe*, 2023. arXiv: 2209.05324 [cs.CV]. URL: <https://arxiv.org/abs/2209.05324>
- [37] Li, Z. i in., *BEVFormer: Learning Bird's-Eye-View Representation from Multi-Camera Images via Spatiotemporal Transformers*, 2022. arXiv: 2203.17270 [cs.CV]. URL: <https://arxiv.org/abs/2203.17270>
- [38] Liao, S., Hu, Y., Zhu, X. i Li, S. Z., „Person Re-identification by Local Maximal Occurrence Representation and Metric Learning”, w: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, s. 2197–2206. DOI: 10.1109/CVPR.2015.7223344

- [39] Lin, T.-Y., Goyal, P., Girshick, R., He, K. i Dollár, P., „Focal Loss for Dense Object Detection”, w: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, s. 2980–2988. DOI: 10.1109/ICCV.2017.324
- [40] Liu, W. i in., „SSD: Single Shot MultiBox Detector”, w: *Computer Vision - ECCV 2016*, seria Lecture Notes in Computer Science, t. 9905, Springer, 2016, s. 21–37. DOI: 10.1007/978-3-319-46448-0_2
- [41] Mallot, H. A., Bülthoff, H. H., Little, J. i Bohrer, S., „Inverse perspective mapping simplifies optical flow computation and obstacle detection”, *Biological cybernetics*, t. 64, nr 3, s. 177–185, 1991.
- [42] Mandeljc, R., Kovačič, S., Kristan, M. i Pers, J., „Tracking by Identification Using Computer Vision and Radio”, *Sensors*, t. 13, s. 241–273, grudzień 2012. DOI: 10.3390/s130100241
- [43] Milan, A., Leal-Taixé, L., Reid, I., Roth, S. i Schindler, K., „MOT16: A Benchmark for Multi-Object Tracking”, *arXiv preprint arXiv:1603.00831*, 2016.
- [44] Mittal, A. i Paragios, N., „Motion-based background subtraction using color, texture and depth”, w: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, t. 2, 2004, s. 233–240.
- [45] Nikouei, M. i in., „Small object detection: A comprehensive survey on challenges, techniques and real-world applications”, *Intelligent Systems with Applications*, t. 27, 2025, ISSN: 2667-3053. DOI: 10.1016/j.iswa.2025.200561 URL: <http://dx.doi.org/10.1016/j.iswa.2025.200561>
- [46] NVIDIA, *The Multi-Camera Tracking AI Workflow*, <https://www.nvidia.com/en-us/ai-data-science/ai-workflows/multi-camera-tracking/>, [Online; accessed 2026-06-24], 2024.
- [47] OpenCV Documentation, *BackgroundSubtractorKNN Class Reference*, OpenCV Developer Manual, 2025.
- [48] OpenCV Team, *OpenCV ChangeLog: Optimizations and Universal Intrinsics*, Dostępne online: <https://github.com/opencv/opencv/wiki/ChangeLog>, 2020.
- [49] Philion, J. i Fidler, S., *Lift, Splat, Shoot: Encoding Images From Arbitrary Camera Rigs by Implicitly Unprojecting to 3D*, 2020. arXiv: 2008.05711 [cs.CV]. URL: <https://arxiv.org/abs/2008.05711>
- [50] Prati, A., Mikic, I., Trivedi, M. M. i Cucchiara, R., „Detecting moving shadows: algorithms and evaluation”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 25, nr 7, s. 799–812, 2003.
- [51] Rao, H. i Wang, S., „TranSG: Transformer-Based Skeleton Graph Prototype Contrastive Learning with Structure-Trajectory Prompted Reconstruction for Person Re-Identification”, w: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, s. 22 118–22 128.

-
- [52] Redmon, J., Divvala, S., Girshick, R. i Farhadi, A., „You Only Look Once: Unified, Real-Time Object Detection”, w: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, s. 779–788. DOI: 10.1109/CVPR.2016.91
- [53] Ren, S., He, K., Girshick, R. i Sun, J., „Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”, w: *Advances in Neural Information Processing Systems 28 (NIPS)*, 2015, s. 91–99. URL: <http://papers.nips.cc/paper/5638-faster-r-cnn-towards-real-time-object-detection-with-region-proposal-networks>
- [54] Ristani, E., Solera, F., Zou, R. S., Cucchiara, R. i Tomasi, C., „Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking”, *arXiv preprint arXiv:1609.01775*, 2016.
- [55] Ryspekov, B., „Performance Analysis of Modern Object Detection Models for Edge-Based Assistive Glasses”, *NHSJS Reports: Computing, Data & AI*, kwiecień 2026.
- [56] Singh, P. R., Gottumukkala, R. i Maida, A., „An Analysis of Kalman Filter based Object Tracking Methods for Fast-Moving Tiny Objects”, *arXiv preprint arXiv:2509.18451v1*, 2025.
- [57] Solovyova, V., „GPU-Accelerated ML/DL Performance on MacBook Pro M5 Pro vs. M4 Max: Feasibility and Benchmarks for Developers.”, *dev.to*, 2026. URL: <https://dev.to/valesys/gpu-accelerated-mldl-performance-on-macbook-pro-m5-pro-vs-m4-max-feasibility-and-benchmarks-for-2ka3>
- [58] Stauffer, C. i Grimson, W., „Adaptive background mixture models for real-time tracking”, w: *Proceedings. 1999 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (Cat. No PR00149)*, t. 2, 1999, 246–252 Vol. 2. DOI: 10.1109/CVPR.1999.784637
- [59] Sun, Y., Zheng, L., Yang, Y. s., Tian, Q. i Wang, S., „Beyond Part Models: Person Retrieval with Refined Part Pooling (and A Strong Convolutional Baseline)”, w: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, s. 480–496.
- [60] Syeda Aynul Karim, *MOG2 background subtraction algorithm process*, https://www.researchgate.net/publication/385367429_Revitalizing_Electoral_Trust_Enhancing_Transparency_and_Efficiency_Throughautomated_Voter_Counting_with_Machine_Learning, data dostępu: 21.05.2026.
- [61] Szeliski, R., *Computer Vision: Algorithms and Applications (2nd Edition)*. Springer, 2022, ISBN: 978-3-030-34372-9.
- [62] Tang, Y., Yang, Z., Xu, Z., Zhou, Y. i Wang, H., „An Improved YOLOv8 Model for Pavement Distress Detection Under Low-Computing-Power Conditions”, *Sensors (Basel)*, t. 26, nr 11, s. 3373, 2026. DOI: 10.3390/s26113373
- [63] Terrell, G. R. i Scott, D. W., „Variable kernel density estimation”, *The Annals of Statistics*, t. 20, nr 3, s. 1236–1265, 1992.
- [64] Viola, P. i Jones, M., „Rapid Object Detection Using a Boosted Cascade of Simple Features”, w: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, t. 1, 2001, s. I-511–I-518. DOI: 10.1109/CVPR.2001.990517

- [65] Waghmare, R., „CHURI: Contour-Gated, Resource-Intelligent Retail Tracking System”, w: *WACV workshop*, Computer Vision Foundation, 2026, s. 1574–1581. URL: https://openaccess.thecvf.com/content/WACV2026W/PRAW/papers/Waghmare_CHURI_Contour-Gated_Resource-Intelligent_Retail_Tracking_System_WACVW_2026_paper.pdf
- [66] Wang, C.-Y., Bochkovskiy, A. i Liao, H.-Y. M., „YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors”, w: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, s. 7464–7475. URL: <https://github.com/WongKinYiu/yolov7>
- [67] Wang, C.-Y., Yeh, I.-H. i Liao, H.-Y. M., „YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information”, *arXiv preprint arXiv:2402.13616*, 2024. URL: <https://github.com/WongKinYiu/yolov9>
- [68] Wang, Y., Jodoin, P.-M., Porikli, F., Konrad, J., Benezeth, Y. i Ishwar, P., „CDnet 2014: An Expanded Change Detection Benchmark Dataset”, *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, czerwiec 2014. DOI: 10.1109/CVPRW.2014.126
- [69] Wille, M., Miller, D., Fischer, T. i Raine, S., *Why Domain Matters: A Preliminary Study of Domain Effects in Underwater Object Detection*, 2026. arXiv: 2604.26174 [cs.CV]. URL: <https://arxiv.org/abs/2604.26174>
- [70] Wojke, N., Bewley, A. i Paulus, D., „Simple online and realtime tracking with a deep association metric (DeepSORT)”, w: *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2017, s. 3645–3649.
- [71] Woo, S., Park, K., Shin, I., Kim, M. i Kweon, I. S., *MTMMC: A Large-Scale Real-World Multi-Modal Camera Tracking Benchmark*, 2024. arXiv: 2403.20225 [cs.CV]. URL: <https://arxiv.org/abs/2403.20225>
- [72] Yang, Z. i in., *Multi-Branch Auxiliary Fusion YOLO with Re-parameterization Heterogeneous Convolutional for accurate object detection*, 2024. arXiv: 2407.04381 [cs.CV]. URL: <https://arxiv.org/abs/2407.04381>
- [73] Zhang, Y. i in., „ByteTrack: Multi-Object Tracking by Associating Every Detection Box”, w: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, s. 1–18.
- [74] Zhang, Z., „A flexible new technique for camera calibration”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 22, nr 11, 2000. DOI: 10.1109/34.888718
- [75] Zhao, J., Shi, J. i Zhuo, L., „BEV Perception for Autonomous Driving: State of the Art and Future Perspectives”, *Expert Systems with Applications*, t. 258, sierpień 2024. DOI: 10.1016/j.eswa.2024.125103
- [76] Zhao, Y. i in., „DETRs Beat YOLOs on Real-time Object Detection”, w: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, s. 16 965–16 974. URL: <https://zhao-yian.github.io/RTDETR>

-
- [77] Zheng, L., Yang, Y. i Hauptmann, A. G., „Person Re-identification: Past, Present and Future”, *arXiv preprint arXiv:1610.02984*, 2016.
- [78] Zheng, L., Zhang, H., Sun, S., Chandraker, M., Yang, Y. i Tian, Q., „Person Re-identification in the Wild”, w: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, s. 1367–1376.
- [79] Zhu, X., Su, W., Lu, L., Li, B., Wang, X. i Dai, J., „Deformable DETR: Deformable Transformers for End-to-End Object Detection”, w: *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. URL: <https://openreview.net/forum?id=gZ9hCDWe6ke>
- [80] Zivkovic, Z., „Improved Adaptive Gaussian Mixture Model for Background Subtraction”, w: *Proceedings - International Conference on Pattern Recognition*, t. 2, wrzesień 2004, 28–31 Vol.2, ISBN: 0-7695-2128-2. DOI: 10.1109/ICPR.2004.1333992
- [81] Zivkovic, Z. i Van der Heijden, F., „Efficient adaptive density estimation per image pixel for the task of background subtraction”, *Pattern Recognition Letters*, t. 27, s. 773–780, maj 2006. DOI: 10.1016/j.patrec.2005.11.005

Wykaz skrótów i symboli

ANN	sztuczna sieć neuronowa (ang. Artificial Neural Network)
CNN	splotowa sieć neuronowa (ang. Convolutional Neural Network)
FLOPS	liczba operacji zmiennoprzecinkowych na sekundę
GPU	procesor graficzny (ang. Graphical Processing Unit)

Spis rysunków

1	Wizualizacja procesu odejmowania tła. Źródło: [60]	6
2	Wizualizacja architektury potoku przetwarzania danych. Źródło: opracowanie własne.	46
3	Ilustracja procesu globalnej asocjacji danych za pomocą grafu dwudzielnego. Algorytm węgierski szuka takich połączeń (pogrubione zielone linie), które minimalizują sumę kosztów C opartych na odległości Re-ID oraz geometrii BEV. Detekcja $D_{c2,2}$ nie spełnia progów podobieństwa i inicjuje nowy identyfikator G_3	50

Spis tabel

1	Średnie wartości współczynnika IoU wybranych algorytmów BGS. Źródło: opracowanie własne na podstawie danych z [31].	11
2	Parametry detektora i trackera lokalnego. Źródło: opracowanie własne.	48
3	Parametry konfiguracyjne modułu analizy tła MOG2 i operacji morfologicznych. Źródło: opracowanie własne.	53

Spis załączników

1	Specyfikacja komputera użytego w pracy	77
2	Dodatkowe zdjęcia badanego obiektu	79

Załącznik 1

Specyfikacja komputera użytego w pracy

Laptop Lenovo YOGA 720-15IKB	
OS	Ubuntu 18.04.4 LTS
OS typ	64-bit
CPU	Intel Core i7-7700HQ CPU @ 2.80GHz x 8
GPU	GeForce GTX 1050/PCIe/SSE2
RAM	16GiB

Załącznik 2

Dodatkowe zdjęcia badanego obiektu

Tu można wstawić dodatkowe zdjęcia, które zajęłyby zbyt dużo miejsca w samej pracy, ale dla lepszej ilustracji tego, co zostało zrobione, można je umieścić w załączniku.